

1.Create Test Image

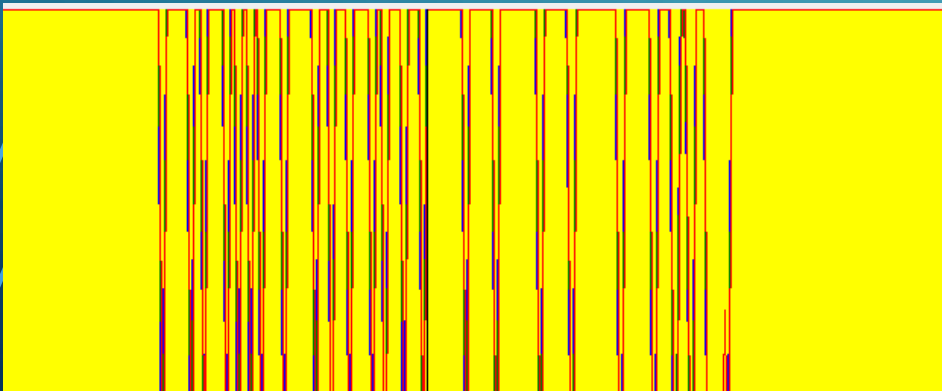
Fig 1.1-Tim1.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

יצרנו את תמונת הטסט Tim1.bmp באמצעות תוכנת Paint.
התמונה היא בגוויני אפור, בגודל של 480X640 פיקסלים,
ומכילה 14 שורות טקסט זהות מהצורה

"Hello from ABCD-EFGH" בגדלים משתנים, החל מגודל
קטן מאוד ועד הגדול במיוחד, כאשר הגדרנו את הגופן הקטן
ביותר ל8 ואת הגדול ביותר ל32.

Fig 1.2



ניתן להבחין ברצף של קפיצות חזקות בין שחור ללבן, דבר
שמעיד על ניגודיות גבוהה.
הקווים החדים מייצגים גבולות של תווים, ונותנים אינדיקציה
לכך שהתמונה מכילה תדירויות גבוהות

2.Creating blurred image Tim1LPFc

על מנת לבצע טשטוש על התמונה Tim1.bmp, השתמשנו במספר פילטרים גאומטריים חד ממדיים כאשר כל אחד מהם בעלי רוחב שונה. בכל ניסוי טענו מחדש את התמונה המקורית, יישמנו עליה את הפילטר באמצעות קונבולוציה אופקית ואנכית (הפונקציה נקראת בשם **DoGaussianFiltration**), ולאחר מכן שמרנו את התוצאה. קבענו את ה HalfSize להיות בגודל של 3.

לקח לנו לבצע 5 גרסאות עד שהגענו לפילטר העמוד בדרישות המשימה שהן: לפחות 2 שורות יהיו קשות לקריאה ו3 אינן ניתנות לקריאה.

Fig 2.1-lpf1 Blur.bmp



Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

בתמונה זו יישמנו פילטר גאומטרי חד ממדי עם $\sigma = 0.8$, שמוגדר על ידי הווקטור הבא:

{ 0.00044, 0.02191, 0.22831, 0.49868, 0.22831, 0.02191, 0.00044 }.

מדובר בפילטר חד ומרוכז מאוד סביב המרכז, ולכן ההשפעה שלו על התמונה מינימלית. זהו בעצם הניסיון הראשון שלנו ורצינו לראות האם זה מספיק על מנת להשפיע על השורות הקטנות בתמונה, אך בפועל קל לראות כי כל השורות, קריאות וברורות.

לכן סינן זה לא מספיק, ונדרש פילטר גאומטרי רחב יותר.

הערה חשובה:

ככל שה σ גדול יותר כל הפילטר פרוש על פני טווח רחב יותר, מה שמעיד על טשטוש חזק יותר.

2.Creating blurred image Tim1LPFc-continuation

Fig 2.2-lpf2Blur.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

בתמונה הזו יישמנו פילטר גאوسي חד ממדי עם $\sigma = 1$, שמוגדר על ידי הווקטור הבא:

{ 0.00443, 0.05401, 0.24204, 0.39905, 0.24204, 0.05401, 0.00443 }.

מדובר בפילטר מעט רחב יותר לעומת הניסוי הקודם, ולכן ציפינו להשפעה משמעותית יותר על חדות התמונה. אכן ניתן להבחין כי חלה ירידה קלה בחדות של השורות הקטנות ביותר. עם זאת, גם לאחר הסינון הזה, כל השורות כולל הקטנות ביותר נותרו קריאות.

המסקנה היא שטשטוש ברמת $\sigma = 1.0$ אינו מספק, ויש לנסות טשטוש חזק יותר בעזרת פילטר בעל רוחב גדול יותר.

הערה בקשר לhalfSize:

halfSize הינו חצי האורך של הפילטר מצד אחד בלבד של המרכז.

אצלנו כל הפילטרים הינם בגודל 7, ועבור תמונה זו הפילטר הינו:

{ 0.00443, 0.05401, 0.24204, 0.39905, 0.24204, 0.05401, 0.00443 }

כפי שניתן לראות שמהאיבר האמצעי שהינו 0.39905 ישנם 3 ערכים מימין ו3 משמאל.

2.Creating blurred image Tim1LPFc-continuation

Fig 2.3-lpf3Blur.bmp



Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

בתמונה הזו יישמנו פילטר גאوسي חד ממדי עם $\sigma = 1.2$, שמוגדר על ידי הווקטור הבא:

{ 0.01465 , 0.08312 , 0.23556 , 0.33335 , 0.23556 , 0.08312 , 0.01465 }.

מדובר בפילטר מעט רחב יותר לעומת הניסוי הקודם, ולכן הוא מטשטש את התמונה בצורה אגרסיבית יותר. ניתן להבחין כי בעוד הטקסט בגופנים הגדולים נותר קריא, ה-3 שורות הקטנות בתחתית התמונה טושטשו ברמה חזקה, אך חלקן ניתנות לקריאה.

המסקנה היא שטשטוש ברמת $\sigma = 1.2$ אינו מספק, ויש לנסות טשטוש חזק יותר עם פילטר רחב יותר על מנת לטשטש שורות כנדרש במשימה.

2.Creating blurred image Tim1LPFc-continuation

Fig 2.4-lpf4Blur.bmp



Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

בתמונה הזו יישמנו פילטר גאوسي חד ממדי עם $\sigma = 1.5$, שמוגדר על ידי הווקטור הבא:

{ 0.03663 , 0.11128 , 0.21675 , 0.27068 , 0.21675 , 0.11128 , 0.03663 }.

מדובר בפילטר רחב יותר לעומת הניסוי הקודם, אשר ממשיך את מגמת הטשטוש המוגבר. כבר במבט ראשוני ניתן לראות כי השפעת הפילטר משמעותית: ה-3 שורות התחתונות כבר מטושטשות לחלוטין ולא ניתנות לקריאה, אך השורות שמעליו מובנות אך מתחילות לאבד מהחדות שלהן.

למרות שהפילטר אינו עומד בדרישות המשימה מבחינת קריאות, ניתן לראות את הקשר בין הגדלת σ לבין הירידה בפרטים הדקים של התמונה.

2.Creating blurred image Tim1LPFc-continuation

Fig 2.5-Tim1LPFc.bmp



Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

בתמונה הזו יישמנו פילטר גאומי חד ממדי עם $\sigma = 3.5$, שמוגדר על ידי הווקטור הבא:

$\{0.11535, 0.14147, 0.15990, 0.16656, 0.15990, 0.14147, 0.11535\}$.

פילטר זה רחב מאוד ביחס לקודמיו, ויוצר טשטוש כבד במיוחד על כל מרכיבי התמונה. ניתן להבחין כי כלל השורות בתמונה, כולל הגדולות והברורות ביותר, איבדו חדות בצורה משמעותית.

השורות התחתונות לא ניתנות לקריאה כלל (לפחות 3), והשורות באמצע ניתנות לקריאה אך עם קושי רב (לפחות 2), בכך הפילטר הנל עומד בדרישות המשימה.

3.HIGH FREQUENCY ENHANCING CONVOLUTION FILTERS

- כעת ננסה לשחזר פרטים שאבדו מהתמונה המטושטשת (Tim1LPFc.bmp), בעזרת פילטרים מרחביים מסוג High Pass.
- השתמשנו ב-5 מסנני חידוד מרחביים כאשר 4 מהם בגודל 3 על 3 והחמישי בגודל 5 על 5, נבחן מספר גרסאות לכל פילטר, כאשר לכל פילטר נבדוק את ההשפעה שלו עם ובלי נירמול (Min-Max + Threshold).
- לאחר מכאן נשמור את התמונות המשוחזרות לפי עוצמות שונות של סינון.

3.High Frequency Enhancing Convolution Filters-

זהו פילטר חידוד קיצוני.

פילטר סימטרי בגודל 3 על 3 כאשר הערך המרכזי שלו הינו 17, ושאר הערכים הינם -2, הפילטר מדגיש שינויים חדים ברמת הבהירות, כלומר, הוא מחדד קצוות ומבליט גבולות בין אזורים בתמונה.

סכום המשקלים הינו 1, מה שאומר שהוא גם מדגיש את הגבולות וגם שומר על בהירות כללית.

פילטר מספר 1

$$\begin{bmatrix} -2 & -2 & -2 \\ -2 & 17 & -2 \\ -2 & -2 & -2 \end{bmatrix}$$

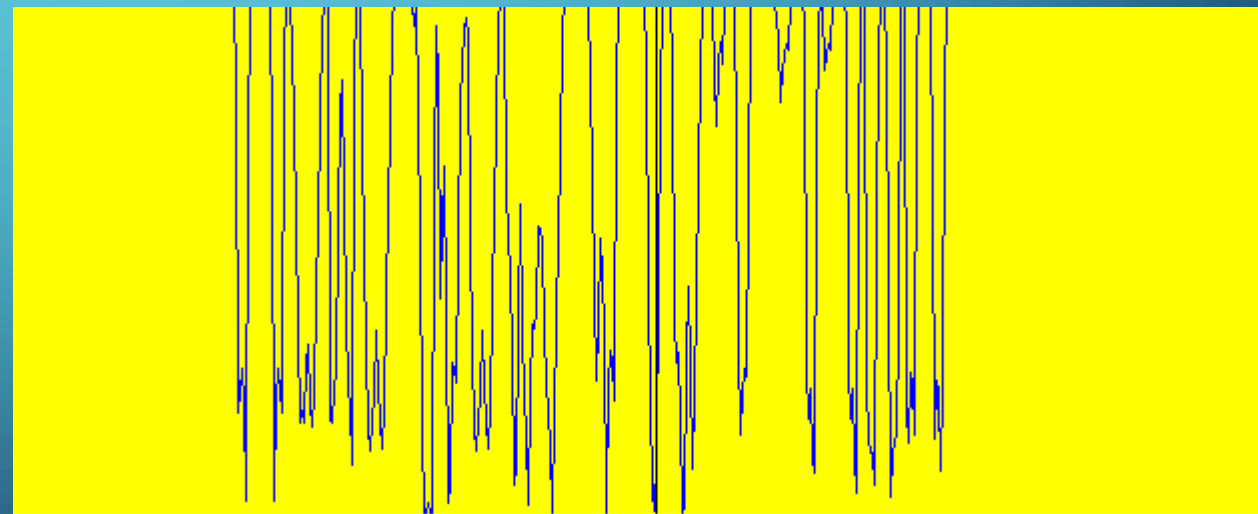


זהו הפילטר בו השתמשנו:

Row profile of fig 3.1

Fig 3.1-HPF1.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

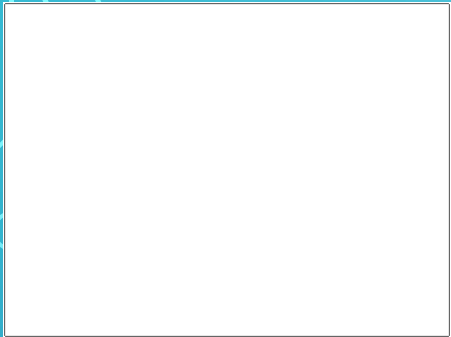


תוצאה של קונבולוציה רגילה ללא סף ו/או נרמול.
הטשטוש הוסר חלקית

ניתן לראות תנודות חזקות, תוצאה אופיינית לפילטר שמדגיש שינויים חדים ברמת הבהירות. הערכים הקיצוניים מצביעים על קצוות ברורים בתמונה.

3. High Frequency Enhancing Convolution Filters-continuation

Fig 3.2-HPF1_1 MINMAX.bmp



ביצענו נרמול MinMax עם סף של 50. ניתן לראות בבירור שהתמונה הפכה ללבנה וכל הכיתוב נמחק

אנו לא רואים בתמונה תנודות ברורים הנראות לעין, בכך ניתן להסיק כי סף הסינון היה גבוה מדי ביחס לערכי הפילטר בפועל, וכתוצאה מכך רוב הערכים הוסרו כ"רעש".

Row profile of fig 3.2

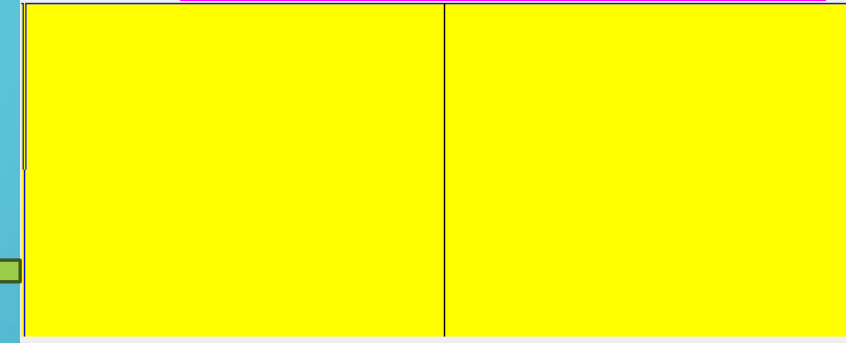


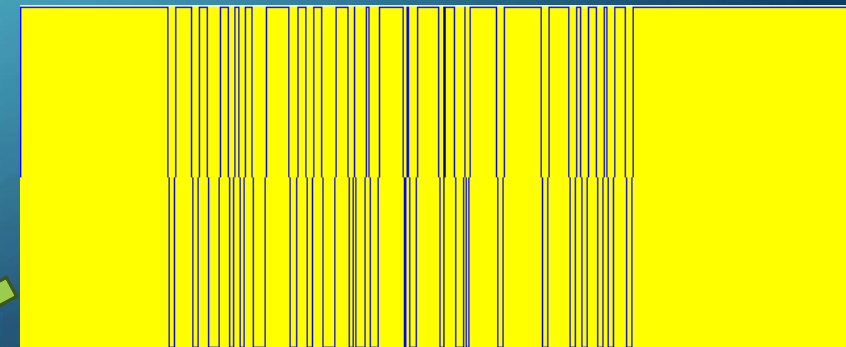
Fig 3.3-HPF1_2MINMAX.bmp



ביצענו נרמול MinMax עם סף של 0.95. ניתן לראות בבירור בשורות העליונות שיפור משמעותי והם מתחדדים ונראים מעולה ביחס לתמונה שעבדנו עליה. עם זאת ניתן להבחין שבשורות התחתונות, חלק מהמידע נמחק ונראה מטושטש יותר.

הגֵּרֶף מציג תנודות חזקות ועקביות באזור המרכזי של התמונה, המעידות על שחזור מוצלח של פרטים חדים בשורות העליונות. האזורים השטוחים בקצוות מצביעים על סינון יתר בשורות התחתונות, שם סף הסינון גרם להיעלמות פרטים עדינים.

Row profile of fig 3.3



3.High Frequency Enhancing Convolution Filters-continuation

Fig 3.4-HPF1_3MINMAX.bmp



ביצענו נרמול MinMax עם סף של 101. ניתן לראות בבירור שהתמונה נהייתה חדה יותר וברורה יותר בשורות העליונות והפונט נהייה מודגש, אך ניכר כי בשורות התחתונות חלק מהכיתוב התחיל להיעלם והכתב אינו קריא.

ניתן להבחין בתנודות חדות וברורות בשורות העליונות אשר מעידות על חידוד טוב של הקצוות. עם זאת, הערכים בצדי התמונות מה שמתאר את השורות הנמוכות כמעט נעלמות, מה שמרמז על אובדן מידע חלקי שם עקב הסף.

Row profile of fig 3.4

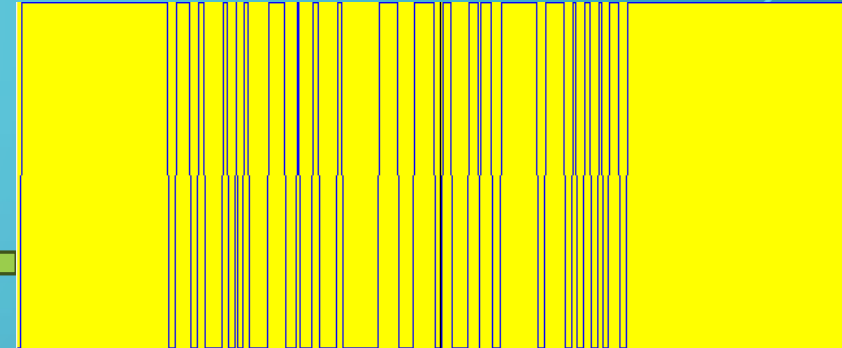


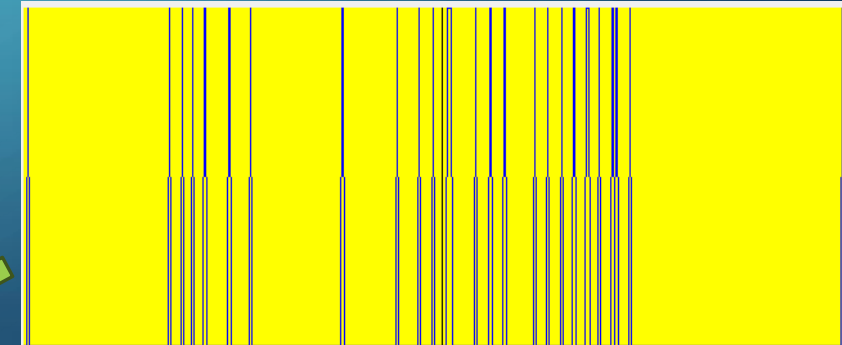
Fig 3.5-HPF1_4MINMAX.bmp



ביצענו נרמול MinMax עם סף של 108. התמונה ממש השתנתה, זה ממש גרם העלמות כמעט מוחלטת של פרטי התמונה – הרקע הפך שחור, הכתב הופיע בצבע לבן ובקונטרסט חזק, אך הפונט איבד צורה והפך קשה לקריאה.

קווים דקים וגבוהים המעידים על שינויים חדים מאוד בין פיקסלים. רוב השטח שטוח, כלומר ערכים קבועים (רקע), בעוד שהקצוות קופצים לערכים גבוהים

Row profile of fig 3.5



3.High Frequency Enhancing Convolution Filters-continuation

זהו פילטר חידוד סימטרי.

פילטר סימטרי בגודל 3 על 3 כאשר הערך המרכזי שלו הינו 9, ושאר הערכים הינם -1, הפילטר מדגיש שינויים חדים ברמת הבהירות, כלומר, הוא מחדד קצוות ומבליט גבולות בין אזורים בתמונה, הוא אידיאלי לחידוד גבולות של אובייקטים ושיפור חדות של טקסטורות.

סכום המשקלים הינו 1, מה שאומר שהוא גם מדגיש את הגבולות וגם שומר על בהירות כללית.

פילטר מספר 2

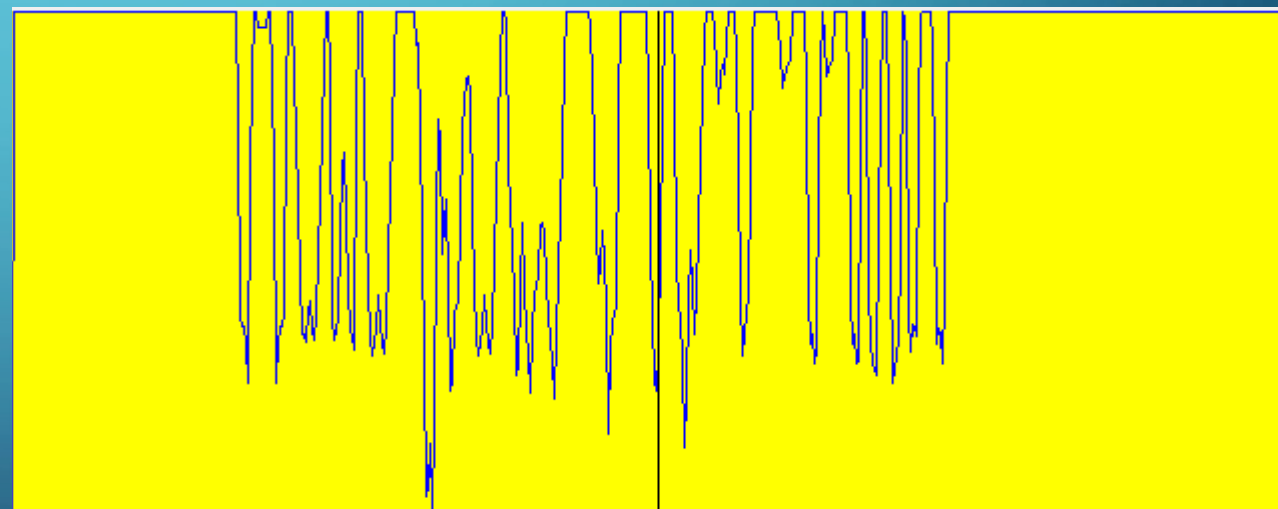
$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$



זהו הפילטר בו השתמשנו:

Row profile of fig 3.6

Fig 3.6-HPF2.bmp



תוצאה של קונבולוציה רגילה ללא סף ו/או נרמול.
הטשטוש הוסר חלקית

ניתן לראות תנודות חזקות, תוצאה אופיינית לפילטר שמדגיש שינויים חדים ברמת הבהירות.

3.High Frequency Enhancing Convolution Filters-continuation

Fig 3.7-HPF2_1 MINMAX.bmp

Row profile of fig 3.7

ביצענו נרמול MinMax עם סף של 50. ניתן לראות בבירור שהתמונה הפכה ללבנה וכל הכיתוב נמחק

אנו לא רואים בתמונה תנודות ברורים הנראות לעין, בכך ניתן להסיק כי סף הסינון היה גבוה מדי ביחס לערכי הפילטר בפועל, וכתוצאה מכך רוב הערכים הוסרו כ"רעש".

Fig 3.8-HPF2_2 MINMAX.bmp

Row profile of fig 3.8

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

ביצענו נרמול MinMax עם סף של 98. ניתן לראות בבירור בשורות העליונות שיפור משמעותי והם מתחדדים ונראים מעולה ביחס לתמונה שעבדנו עליה. עם זאת ניתן להבחין שבשורות האמצעיות, חלק מהמידע נמחק וניהיה מטושטש יותר, ובשורות התחתונות המידע כבר מחוק כמעט לגמרי ולא ניתן להבין כלום.

הגרף מציג תנודות חזקות ועקביות באזור המרכזי של התמונה, המעידות על שחזור מוצלח של פרטים חדים בשורות העליונות. בצדי הגרף, המתארים את השורות התחתונות, העוצמה נחלשת והאות מתרכך. התבנית הזו משקפת את הפגיעה היחסית בפרטים באזורים אלו

3.High Frequency Enhancing Convolution Filters-continuation

Fig 3.9-HPF2_3MINMAX.bmp



ביצענו נרמול MinMax עם סף של 115. התמונה ממש השתנתה, זה ממש גרם העלמות כמעט מוחלטת של פרטי התמונה – הרקע הפך שחור, הכתב הופיע בצבע לבן ובקונטרסט חזק, אך הפונט איבד צורה והפך קשה לקריאה.

גרף הפרופיל מציג תנודות חדות בחלק המרכזי של השורה, המעידות על קצוות בולטים באזור הטקסט. לעומת זאת, באזורים השוליים של הגרף נצפות רמות שטוחות ונמוכות, דבר המצביע על כך שהמידע באזורים אלה נחתך או נחלש

Row profile of fig 3.9

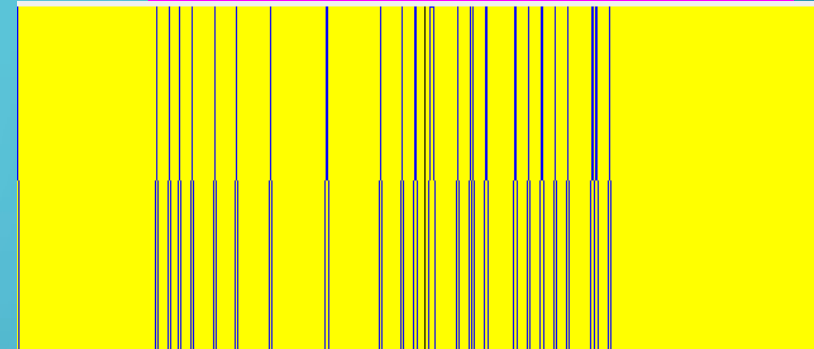


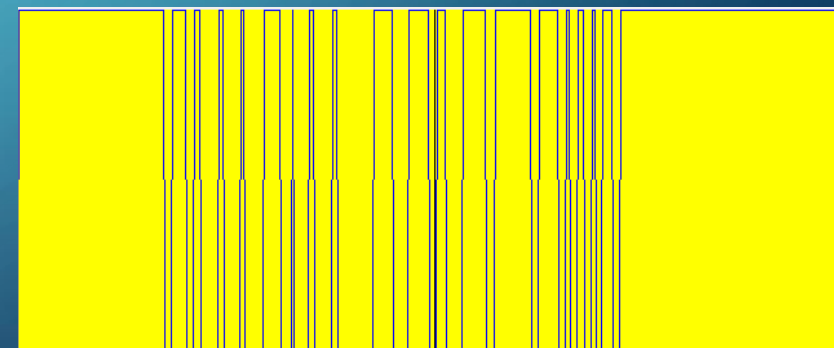
Fig 3.10-HFEF1cTim1LPFc.bmp



ביצענו נרמול MinMax עם סף של 106. התמונה מתארת במדויק את דרישות המשימה. השחזור בוצע באמצעות פילטר HPF1, בשילוב נרמול MinMax עם סף אופטימלי, אשר שימר את הפרטים החשובים והחזיר חדות מלאה לכל שורות הטקסט. בניגוד לניסיונות אחרים, התמונה אינה מציגה עיוותים או מחיקות, ולכן נבחרה כתוצאה הסופית.

קווים דקים וגבוהים המעידים על שינויים חדים מאוד בין פיקסלים. רוב השטח שטוח, כלומר ערכים קבועים (רקע), בעוד שהקצוות קופצים לערכים גבוהים

Row profile of fig 3.10



3.High Frequency Enhancing Convolution Filters-continuation

זהו פילטר חידוד סימטרי ומוגבר.

פילטר סימטרי בגודל 3 על 3 כאשר הערך המרכזי שלו הינו 64, ושאר הערכים הינם -8 או -4, הפילטר נועד להדגיש קצוות בצורה אגרסיבית, תוך הגברה חדה של ההבדלים בין אזורים סמוכים בתמונה.

סכום המשקלים לאחר נרמול של 16 הינו 1, מה שאומר שהוא גם מדגיש את הגבולות וגם שומר על בהירות כללית.

פילטר מספר 3

$$\begin{bmatrix} -4 & -8 & -4 \\ -8 & 64 & -8 \\ -4 & -8 & -4 \end{bmatrix}$$



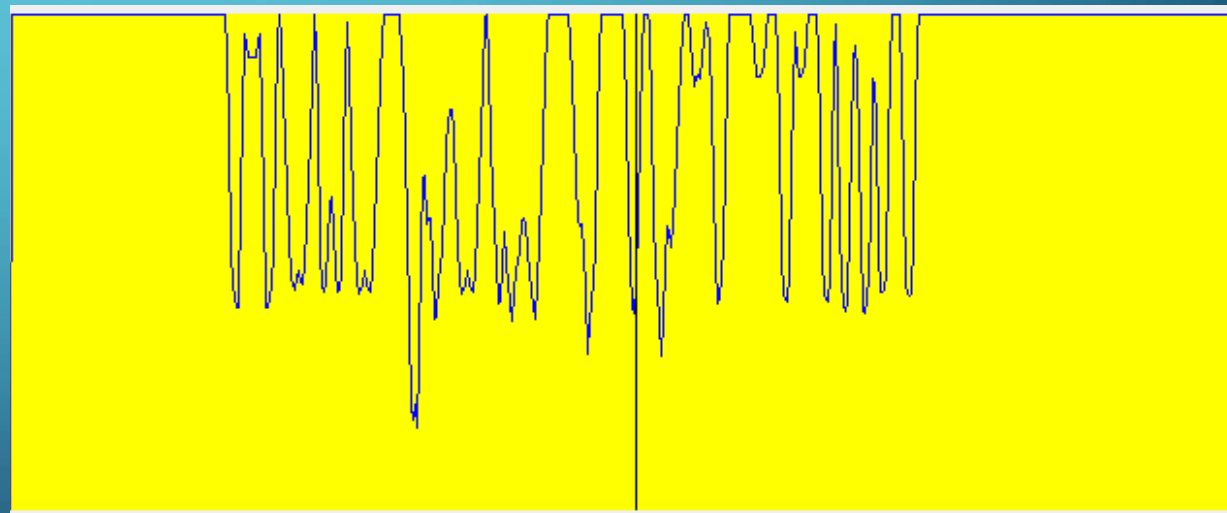
זהו הפילטר בו השתמשנו:

Fig 3.1 1-HPF3.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

התמונה מציגה חידוד קצוות אגרסיבי תוך שמירה על הבהירות הכללית, כל זה בזכות לנרמול הפילטר.
ניתן להבחין בהפחתה חלקית של הטשטוש.

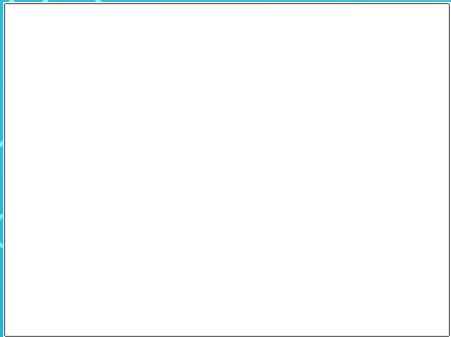
Row profile of fig 3.1 1



גרף הפרופיל מציג תנודות חדות ומשמעותיות, המעידות על חידוד אגרסיבי של הגבולות בתמונה. ערכי הבהירות משתנים בקצב מהיר, מה שמאשש את פעולת הפילטר HPF3 שגורם להבלטה חזקה של קצוות תוך שמירה על מבנה עוצמות הבהירות הכללי הודות לנרמול.

3. High Frequency Enhancing Convolution Filters-continuation

Fig 3.1 2-HPF3_1 MINMAX.bmp



ביצענו נרמול MinMax עם סף של 50. ניתן לראות בבירור שהתמונה הפכה ללבנה וכל הכיתוב נמחק

אנו לא רואים בתמונה תנודות ברורים הנראות לעין, בכך ניתן להסיק כי סף הסינון היה גבוה מדי ביחס לערכי הפילטר בפועל, וכתוצאה מכך רוב הערכים הוסרו כ"רעש".

Row profile of fig 3.12

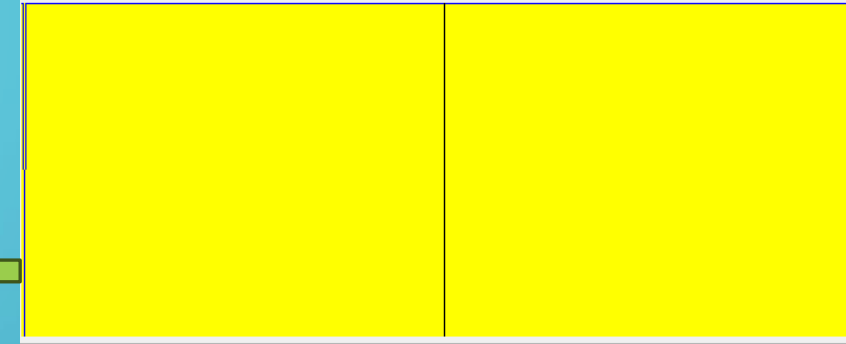


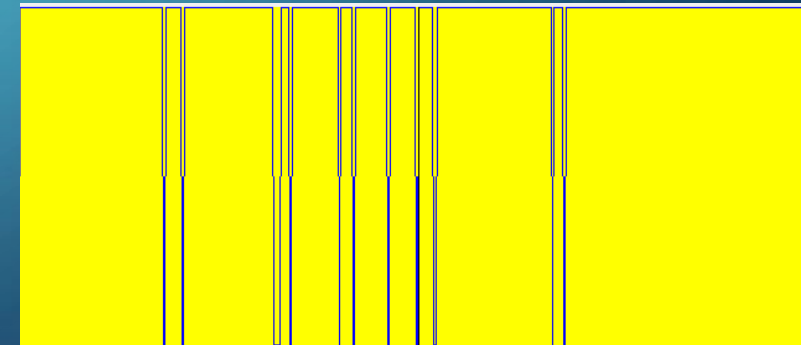
Fig 3.13-HPF3_2MINMAX.bmp



ביצענו נרמול MinMax עם סף של 98. ניתן לראות חידוד משמעותי של השורות העליונות, הכולל הבלטה חדה של גבולות ושיפור ניכר בקריאות הכיתוב. עם זאת ניתן להבחין שהחל מהאמצע התמונה ומטה, חלה ירידה משמעותית באיכות, חלק גדול מהמידע נמחק ונהיה מטושטש יותר ולא ניתן להבין כלום.

הפרופיל לאורך שורות התמונה מציג דפוס ברור של קווים מחודדים באזורים העליונים בלבד, עם פעילות מרוכזת בחלק המרכזי-עליון של הגרף. הדבר תואם את תיאור התמונה עצמה, חידוד מוצלח באזור העליון, בעוד שהאזורים התחתונים שומרים על אחידות.

Row profile of fig 3.13



3.High Frequency Enhancing Convolution Filters-continuation

Fig 3.14-HPF3_3MINMAX.bmp



ביצענו נרמול MinMax עם סף של 115. ניתן לראות חידוד משמעותי של השורות העליונות, אשר כולל הבלטה חדה של גבולות ושיפור ניכר בקריאות הכיתוב. עם זאת, החל מאמצע התמונה ומטה, חלה ירידה באיכות, חלק מהפרטים אובדים, התמונה נהיית מטושטשת, וחלק מהכיתוב אינו קריא כלל.

קווים דקים וגבוהים המעידים על שינויים חדים מאוד בין פיקסלים. רוב השטח שטוח, כלומר ערכים קבועים (רקע), בעוד שהקצוות קופצים לערכים גבוהים

Row profile of fig 3.14

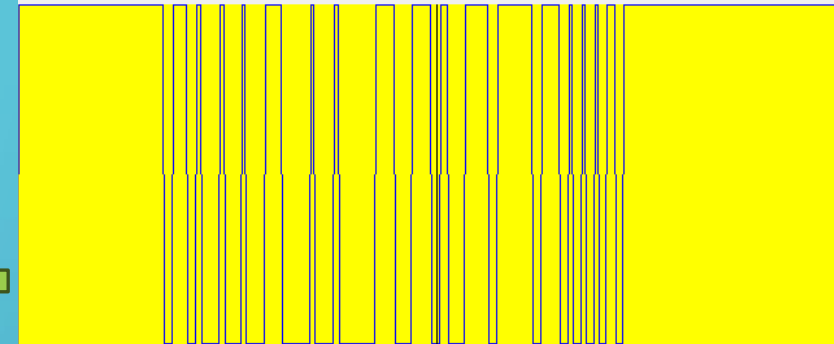


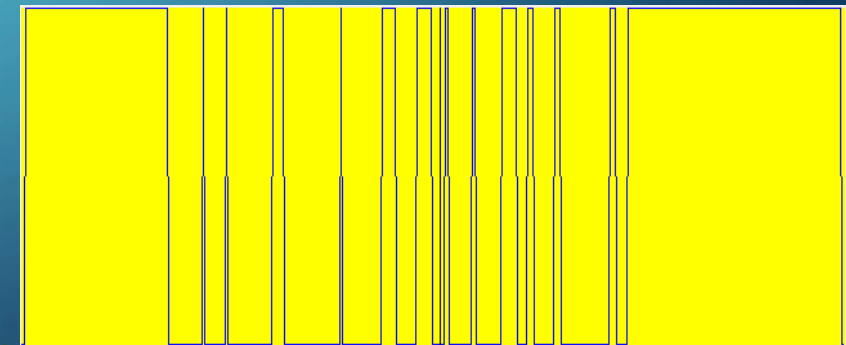
Fig 3.15-HPF3_4MINMAX.bmp



ביצענו נרמול MinMax עם סף של 106. ניתן להבחין בשיפור חלקי באזורים העליונים של התמונה, שורות וכיתובים הפכו חדים ומובחנים יותר. עם זאת, בהשוואה לערכי הסף קודמים, באזורים התחתונים של התמונה נראית הדגשה חזקה מדי שמקשה על פענוח התוכן, והכיתוב נעשה פחות ברור.

ניתן לראות שהגרף מציג תנודות חזקות ברמות הבהירות באזורים מסוימים, עם קווים חדים ודקים המעידים על שינויים פתאומיים בין פיקסלים. האזורים האחידים בתמונה נשארו כמעט אחידים, אבל יש הרבה קפיצות חדות באמצע הגרף, מה שמעיד על חידוד חזק מאוד של הגבולות בתמונה, מה שתואם את התוצאה החזותית.

Row profile of fig 3.15



3.High Frequency Enhancing Convolution Filters-continuation

פילטר מספר 4

זהו הפילטר בו
השתמשנו:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$



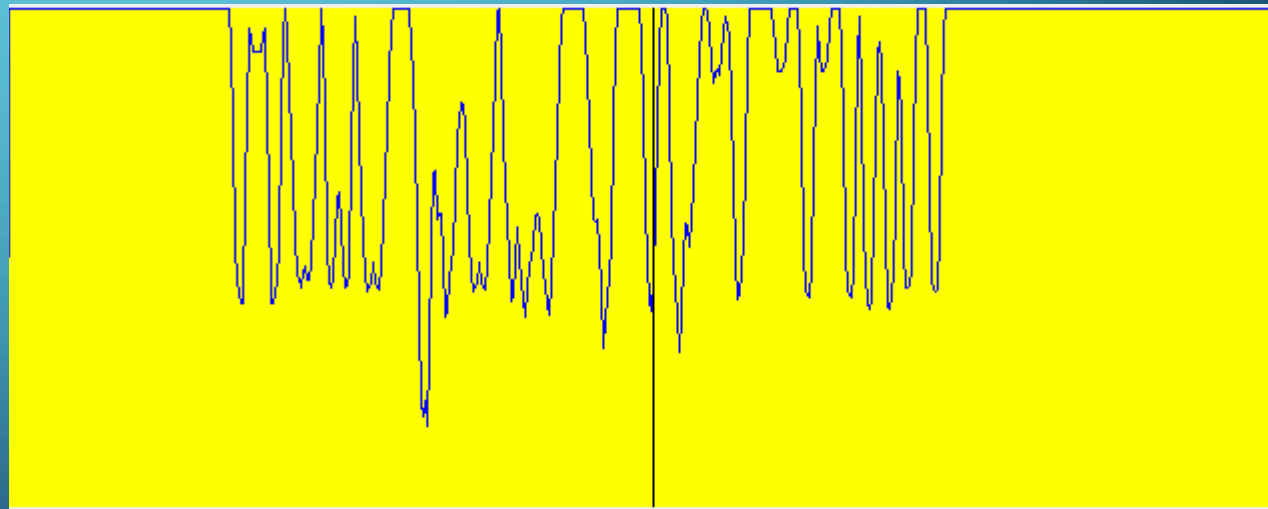
זהו פילטר חידוד עדין.
פילטר סימטרי בגודל 3 על 3 כאשר הערך המרכזי שלו הינו 5, ושאר הערכים הינם -1, מטרת הפילטר היא לחזק את הקצוות במקומות שבהם יש שינוי פתאומי ברמת הבהירות, אך לשמר יותר פרטים.
סכום המשקלים הינו 1, מה שאומר שהוא גם מדגיש את הגבולות וגם שומר על בהירות כללית.

Row profile of fig 3.16

Fig 3.16-HPF4.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

התמונה מציגה חידוד קצוות אגרסיבי תוך שמירה על הבהירות הכללית, כל זה בזכות לנרמול הפילטר.
ניתן להבחין בהפחתה חלקית של הטשטוש.



גרף הפרופיל מציג תנודות חדות ומשמעותיות, המעידות על חידוד אגרסיבי של הגבולות בתמונה. ערכי הבהירות משתנים בקצב מהיר, מה שמאשש את פעולת הפילטר HPF4 שגורם להבלטה חזקה של קצוות תוך שמירה על מבנה עוצמות הבהירות הכללי הודות לנרמול.

3. High Frequency Enhancing Convolution Filters-continuation

Fig 3.17-HPF4_1MINMAX.bmp

[illegible]

ביצענו נרמול MinMax עם סף של 0.88. ניתן לראות שיפור חלקי בלבד בשורות העליונות, שם חלק מהכיתוב עבר חידוד מסוים ונעשה ברור יותר. לעומת זאת, בשורות התחתונות התמונה נותרה מטושטשת, והכיתוב אינו קריא.

ברוב חלקי התמונה לא נרשמו שינויים בבהירות. יחד עם זאת, מופיעות מספר פסגות חדות בצפיפות נמוכה, דבר המעיד על הדגשה מתונה של גבולות באזורים בודדים בלבד.

Row profile of fig 3.17

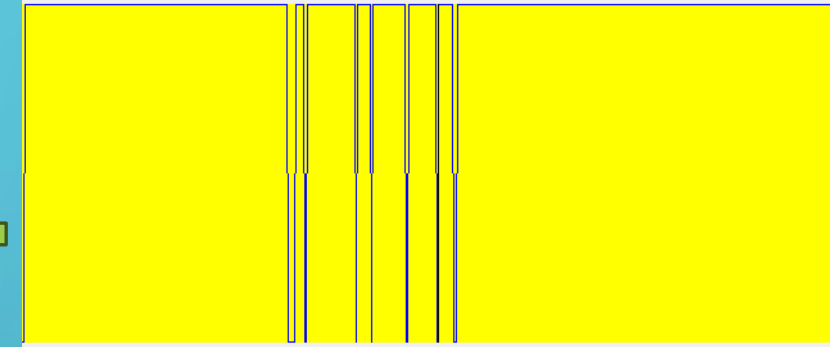
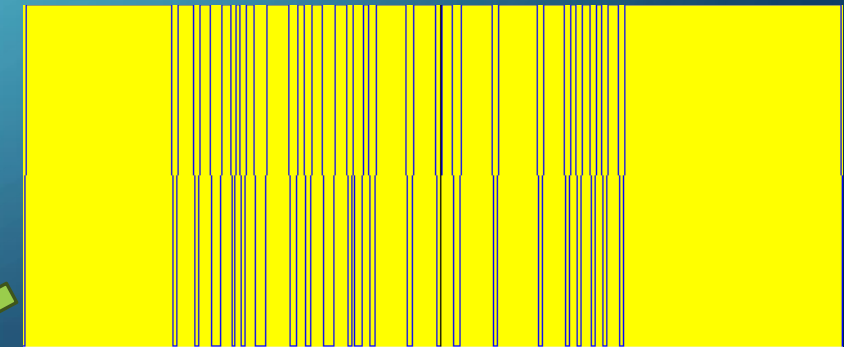


Fig 3.18-HPF4_2MINMAX.bmp

ביצענו נרמול MinMax עם סף של 95. ניתן לראות שיפור ברור בחידוד התמונה לעומת הסף הקודם. הכיתובים והקווים הדקים בשורות העליונות חדים וברורים יותר, והחלקים המרכזיים בתמונה מתחילים להיות קריאים. יחד עם זאת, עדיין ניכרת ירידה הדרגתית באיכות באזורים התחתונים (אך טובה יותר מהתמונה הקודמת)

קווים דקים וגבוהים המעידים על שינויים חדים מאוד בין פיקסלים. רוב השטח שטוח, כלומר ערכים קבועים (רקע), בעוד שהקצוות קופצים לערכים גבוהים

Row profile of fig 3.18



3.High Frequency Enhancing Convolution Filters-continuation

Fig 3.19-HPF4_3MINMAX.bmp



ביצענו נרמול MinMax עם סף של 108. השורות העליונות חודדו ונראות ברורות וחדות, אך בשורות התחתונות ניכר אובדן מידע, חלק מהכיתוב הפך למטושטש או נעלם בגלל חידוד יתר.

ניתן להבחין בריבוי קפיצות חדות בצפיפות גבוהה, בעיקר במרכז, דבר המצביע על הדגשה חזקה של גבולות. הרקע נותר יחסית שטוח, אך הפסגות הרבות מלמדות על שינוי פתאומי בערכי הבהירות.

Row profile of fig 3.20

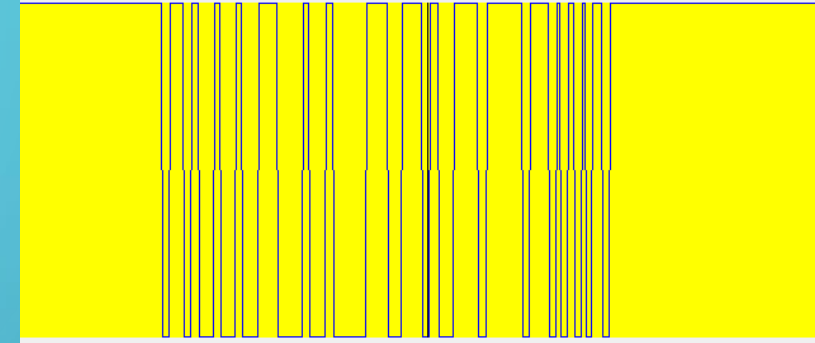


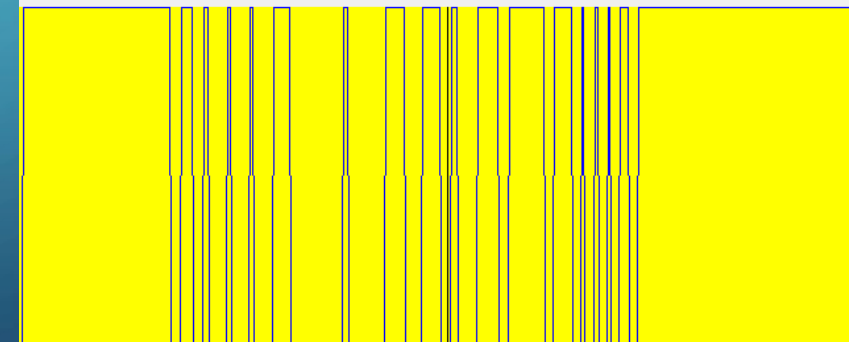
Fig 3.20-HPF4_4MINMAX.bmp



ביצענו נרמול MinMax עם סף של 116. ניתן להבחין בהדגשה חזקה מאוד של גבולות הטקסט, בעיקר בשורות העליונות, שם האותיות נראות חדות וברורות. עם זאת, בשורות האמצעיות והתחתונות, עוצמת הסף יוצרת ניגודיות גבוהה מדי שגורמת לאיבוד פרטים, הן מאוד מטושטשות כך שלא ניתן להבין מה רשום.

בגרף ניתן לראות שהערכים בצדדים נותרו יציבים ואחידים לאורך זמן, מה שמעיד על אזורים חלקים בתמונה שבהם לא התרחשו שינויים חזותיים משמעותיים. לעומת זאת, במרכז הגרף נצפו שיאים חדים וצפופים המעידים על חידוד קצוות והבלטה של גבולות בין אזורים בתמונה, בהתאם לפעולת הפילטר.

Row profile of fig 20



3.High Frequency Enhancing Convolution Filters-continuation

זהו פילטר חידוד חזק מאוד.

פילטר סימטרי בגודל 5 על 5 כאשר הערך המרכזי שלו הינו 25, ושאר הערכים הינם 1-, הפילטר בודק הבדל גדול מאוד בין הפיקסל המרכזי לסביבתו הקרובה, ובכך מדגיש קצוות חזקים במיוחד.

סכום המשקלים הינו 1, מה שאומר שהוא גם מדגיש את הגבולות וגם שומר על בהירות כללית.

פילטר מספר 5

זהו הפילטר בו השתמשנו:

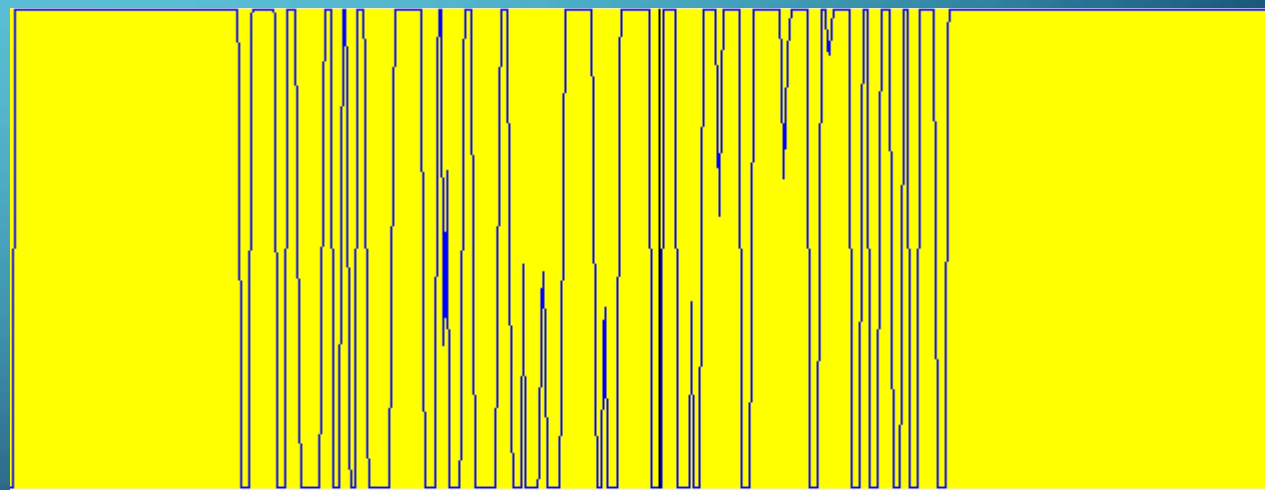
$$\begin{bmatrix} -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & 25 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 & -1 \end{bmatrix}$$

Row profile of fig 3.21

Fig 3.21-HPF5.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
~~Hello from 1753-2289~~
~~Hello from 1753-2289~~
~~Hello from 1753-2289~~
~~Hello from 1753-2289~~

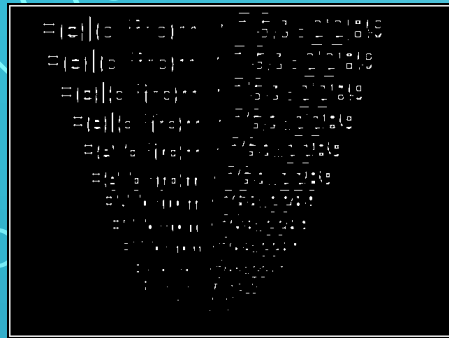
תן לראות הדגשה ברורה של גבולות וקצוות חזקים
בתמונה, אך בצורה פחות "רועשת" מאשר
בפילטרים קודמים, הטשטוש הוסר חלקית



ניתן לראות תנודות חזקות, תוצאה אופיינית לפילטר שמדגיש שינויים חדים ברמת הבהירות. הערכים הקיצוניים מצביעים על קצוות ברורים בתמונה.

3.High Frequency Enhancing Convolution Filters-continuation

Fig 3.24-HPF2_3MINMAX.bmp



ביצענו נרמול MinMax עם סף של 120. התמונה ממש השתנתה, זה ממש גרם העלמות כמעט מוחלטת של פרטי התמונה – הרקע הפך שחור, הכתב הופיע בצבע לבן ובקונטרסט חזק, ולא ניתן להבין כלום ממה שרשום

מתקבלות תנודות חזקות אך מדוללות יותר לאורכו, עם קווים דקים ומרווחים יחסית המייצגים אזורי מעבר בתמונה. לעומת גרפים עם ספים נמוכים יותר, כאן ניכרת הפחתה בכמות הגבולות המודגשים, חידוד הגבולות נותר מורגש.

Row profile of fig 3.24

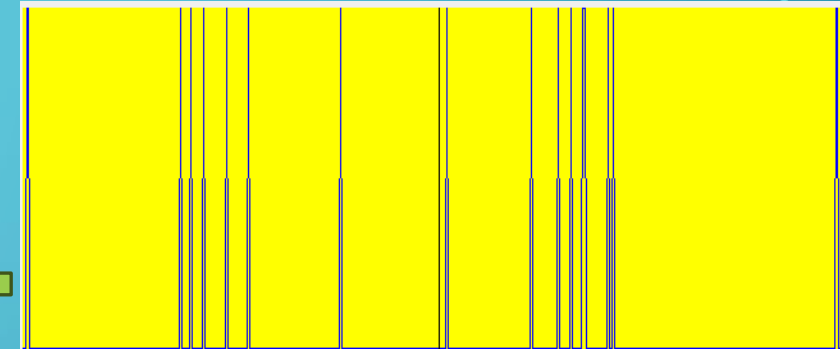


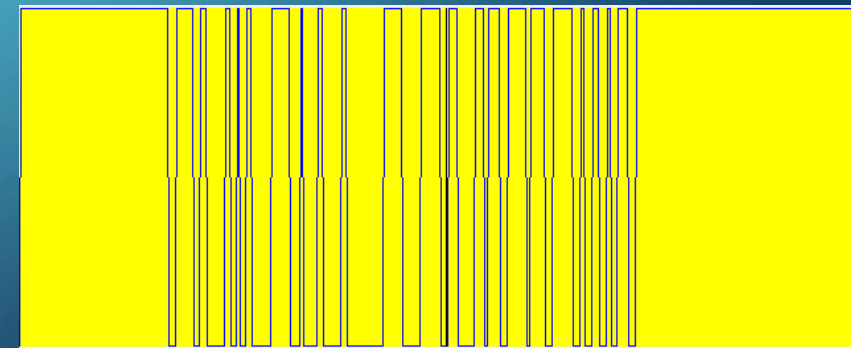
Fig 3.25-HFEF1cTim1 LPFc.bmp



ביצענו נרמול MinMax עם סף של 102. התמונה מתארת במדויק את דרישות המשימה. השחזור בוצע באמצעות פילטר HPF5, בשילוב נרמול MinMax עם סף אופטימלי, אשר שימר את הפרטים החשובים והחזיר חדות מלאה לכל שורות הטקסט. בניגוד לניסיונות אחרים, התמונה אינה מציגה עיוותים או מחיקות, ולכן נבחרה כתוצאה הסופית.

קווים דקים וגבוהים המעידים על שינויים חדים מאוד בין פיקסלים. רוב השטח שטוח, כלומר ערכים קבועים (רקע), בעוד שהקצוות קופצים לערכים גבוהים

Row profile of fig 3.25



4.FILTRATION WITH FFT TO BLUR TIM1.BMP

Fig 4.1-Tim1.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

Row profile of fig 4.1

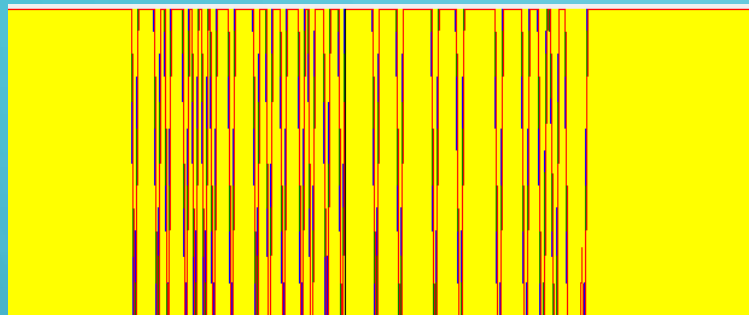


Fig 4.2-gaussFilter100.bmp

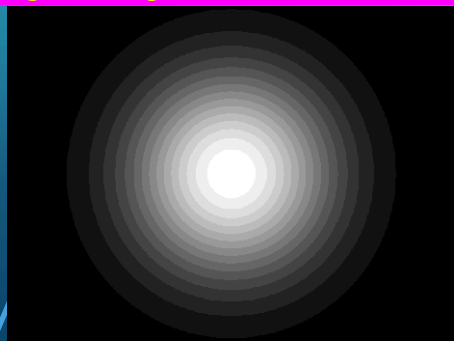
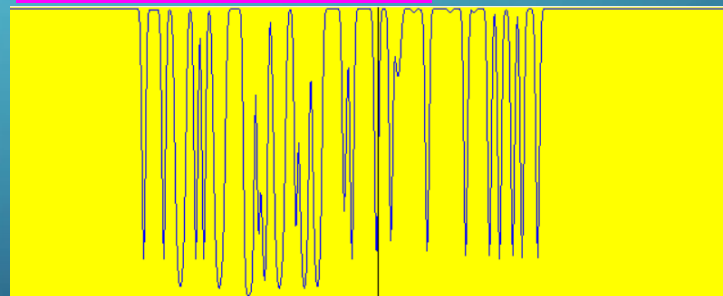


Fig 4.3-BlurredFFT1.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

Row profile of fig 4.3



התמונה המקורית עברה טשטוש בתחום התדר, על ידי מסנן גאוס של רדיוס של 100, מדובר בטשטוש עדין יחסית, כך שניתן להבחין שהטקסט עדיין קריא בכל השורות, כולל הקטנות ביותר. עם זאת, רמת החדות הכללית ירדה מעט, בעיקר באזורים התחתונים, שם נראית פגיעה מסוימת בפרטים הקטנים.

בהתייחסות לתמונת הפרופיל, בתמונה המקורית ניתן לראות שיאים חדים ומובהקים, ולאחר הטשטוש הפרופיל נעשה רך ומתון יותר, עם ירידה בעוצמת השיאים והמעברים. דבר הגרם להשטחת הקצוות ואובדן פרטים חדים בתמונה.

4.filtration with FFT to blur Tim1.bmp-continuation

Fig 4.4-gaussFilter80.bmp

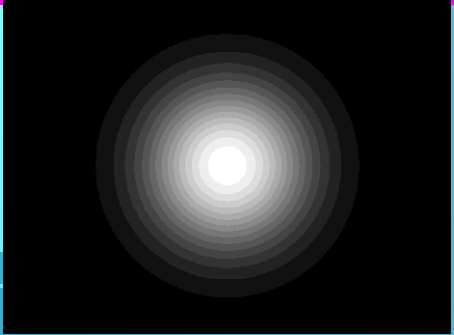
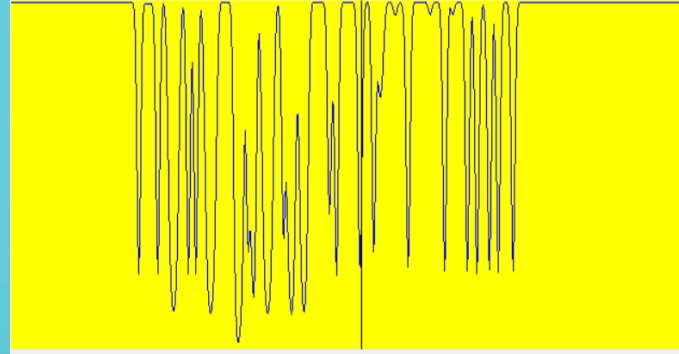


Fig 4.5-BlurredFFT2.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

Row profile of fig 4.5



כעת השתמשנו מסנן גאוס' עם רדיוס של 80, ניתן להבחין שיש שהטשטוש כמעט לא וקרה, חדות התמונה טיפה ירדה אך כל השורות מובנות וברורות. הפרופיל נעשה רך ומתון עוד יותר, עם ירידה בעוצמת השיאים והמעברים, דבר הגרם להשטחת הקצוות ואובדן פרטים חדים בתמונה

Fig 4.6-gaussFilter70.bmp

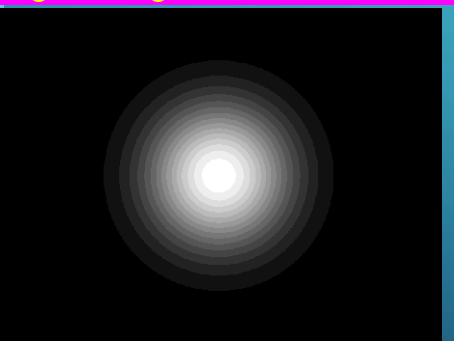
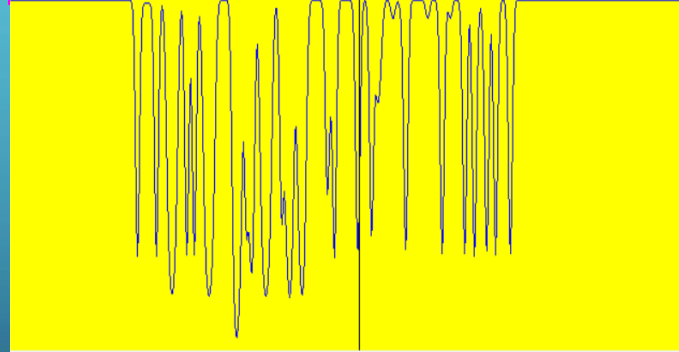


Fig 4.6-BlurredFFT3.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

Row profile of fig 4.6



כעת השתמשנו מסנן גאוס' עם רדיוס של 70 הטשטוש טיפה מתחיל להיות מורגש, כך שהשורה התחתונה מעט מטושטשת, אך עדיין ניתן לקרוא את שאר הטקסט בצורה מלאה. הפרופיל נעשה רך ומתון עוד יותר, עם ירידה בעוצמת השיאים.

4.filtration with FFT to blur Tim1.bmp-continuation

Fig 4.7-gaussFilter60.bmp

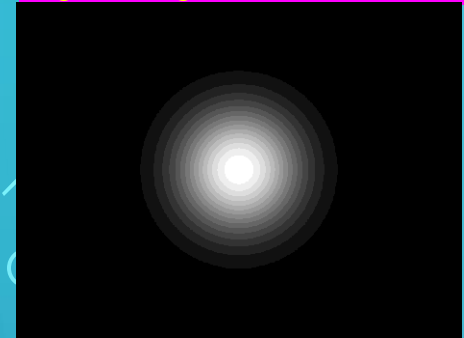
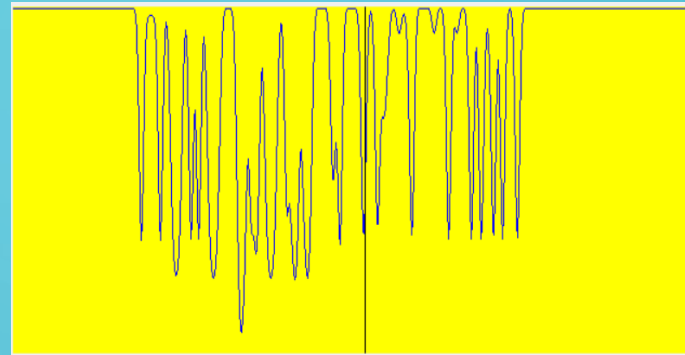


Fig 4.8-BlurredFFT4.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

Row profile of fig 4.8



כעת השתמשנו מסנן גאוס' עם רדיוס של 60, הטשטוש הפך למשמעותי הרבה יותר, אנחנו התקרבו בניסיון זה משמעותית לפתרון המשימה, וזה בעיקר ב-3 השורות, למרות זאת, שאר הטקסט נראה ברור. הפרופיל נעשה רך ומתון עוד יותר, עם ירידה בעוצמת השיאים.

Fig 4.9-gaussFilter50.bmp

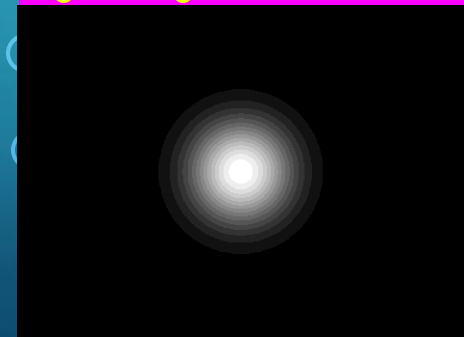
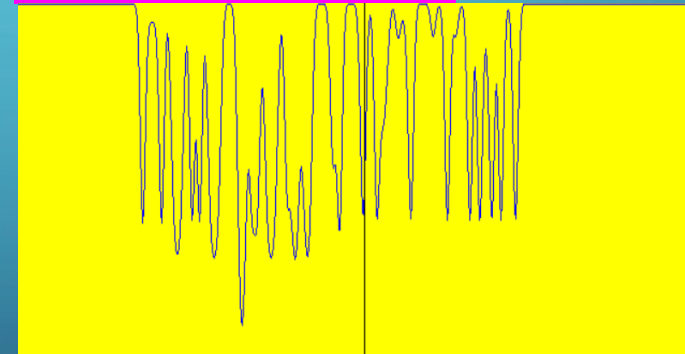


Fig 4.10-BlurredFFT5.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

Row profile of fig 4.10



כעת השתמשנו מסנן גאוס' עם רדיוס של 50, הטשטוש הפך למשמעותי הרבה יותר, למרות זאת, עדיין ניתן לקרוא חלק מהטקסט, אך רמת החדות ירדה משמעותית. הפרופיל נראה זהה לקודמו, אך עם ירידה קלה בעוצמת השיאים. זוהי עדיין לא תוצאה רצויה לכן נקטין את הרדיוס עוד יותר.

4.filtration with FFT to blur Tim1.bmp-continuation

Fig 4.11-gaussFilter40.bmp

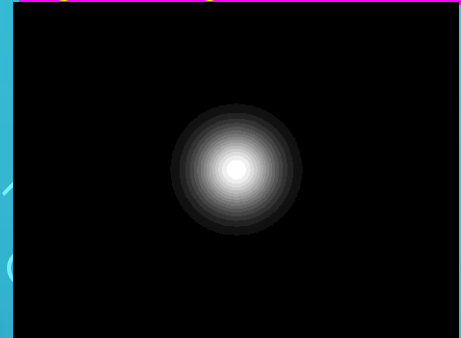
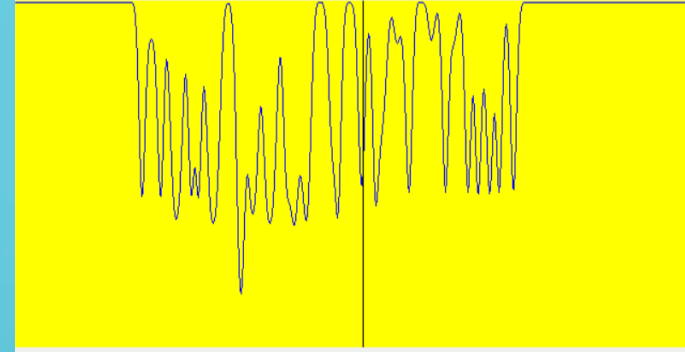


Fig 4.12-BlurredFFT6.bmp



Row profile of fig 4.12



כעת השתמשנו מסנן גאוס' עם רדיוס של 40, הטשטוש הפך למשמעותי הרבה יותר, אנחנו התקרבו בניסיון זה משמעותית לפתרון המשימה. הפרופיל נעשה רך ומתון עוד יותר, עם ירידה בעוצמת השיאים.

Fig 4.13-gaussFilter32.bmp

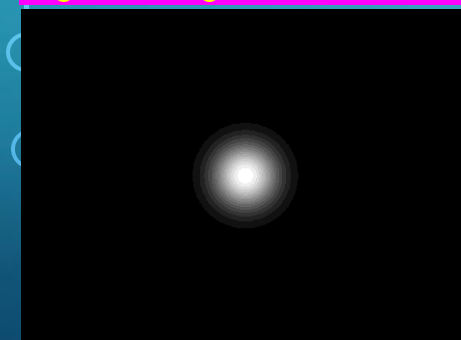
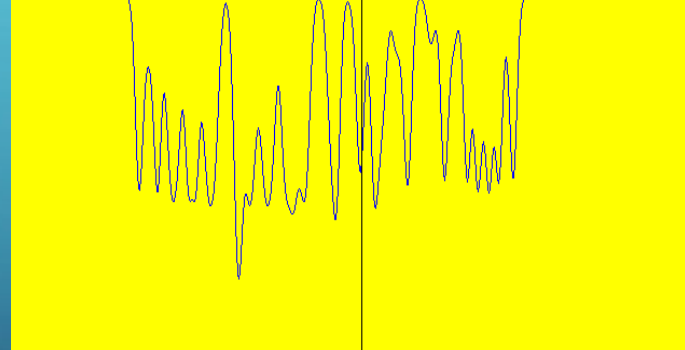


Fig 4.14-BlurredFFT7.bmp



Row profile of fig 4.14



כעת השתמשנו מסנן גאוס' עם רדיוס של 32, הטשטוש הפך למשמעותי הרבה יותר, ניכר כי זהו הפתרון הסופי של המשימה, בכך שזה מתאר הפרופיל נראה זהה לקודמו.

4. filtration with FFT to blur Tim1.bmp-continuation

Fig 4.15-gaussFilter30.bmp

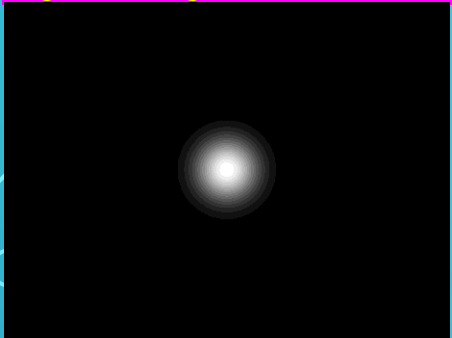
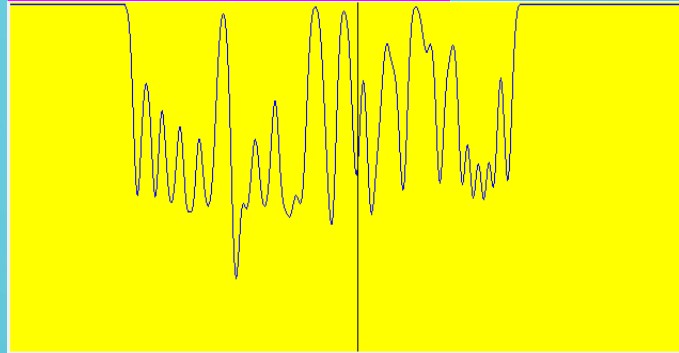


Fig 4.16-BlurredFFT8.bmp



Row profile of fig 4.16



כעת השתמשנו מסנן גאוס' עם רדיוס של 30, הטשטוש הינו טיפה יותר גדול בתמונה הקודמת. הפרופיל נעשה רך ומתון עוד יותר, עם ירידה בעוצמת השיאים. אך עבור רדיוס 32 (המינימלי ביותר) אנחנו עומדים בדרישות המשימה. כלומר אפשר להקטין ולטשטש עוד אבל 32 עושה לנו עבודה מעולה.

Fig 4.17-gaussFilter20.bmp

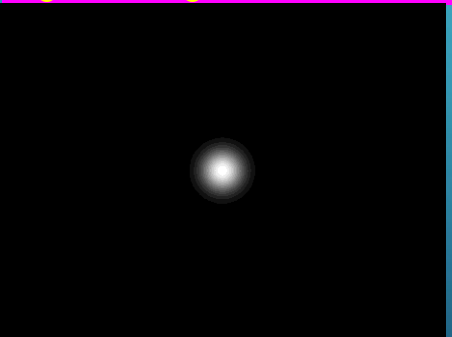
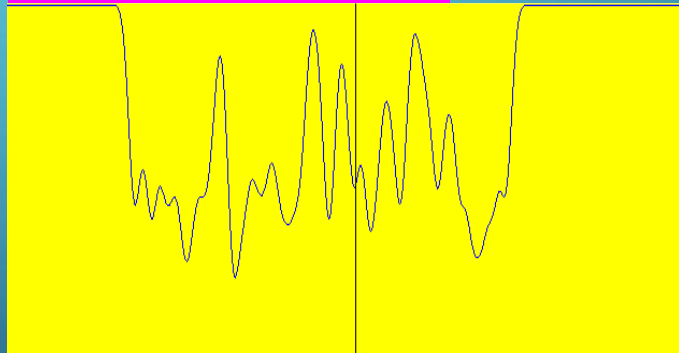


Fig 4.18-BlurredFFT9.bmp



Row profile of fig 4.18



כעת השתמשנו מסנן גאוס' עם רדיוס של 20, הטשטוש הפך למשמעותי הרבה יותר כך שלא ניתן להבין מה כתוב בכל שורה. הפרופיל נעשה חלק בהרבה יותר בצורה משמעותית. אך עבור רדיוס 32 (המינימלי ביותר) אנחנו עומדים בדרישות המשימה.

אנחנו יודעים שהשקופית הזו לא הייתה נדרשת במשימה (כי 32 הינו טשטוש מצוין, העומד בדרישות המשימה) אך רצינו לפרט ולהרחיב על שלבי הפעולה עד שהגענו לתוצאה הרצויה, לכן הוספנו את התמונות הללו.

4.filtration with FFT to blur Tim1.bmp-continuation

Fig 4.13-gaussFilter32.bmp

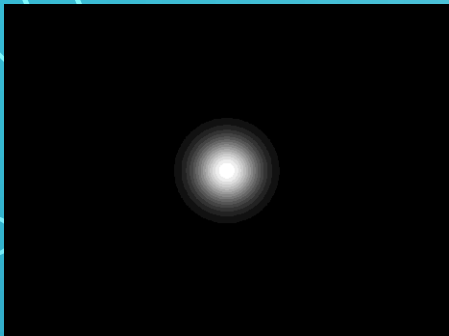
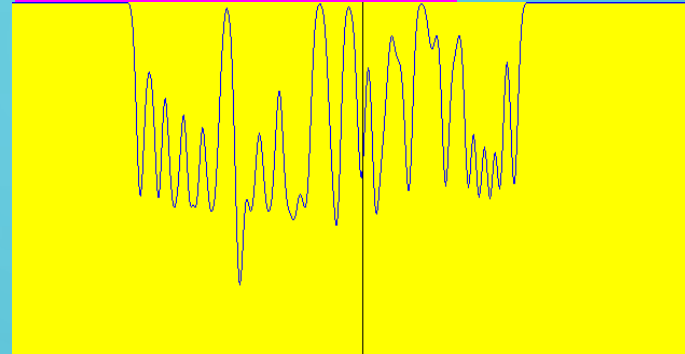


Fig 4.14-BlurredFFT7.bmp



Tim1LPFfft.bmp

Row profile of fig 4.14



מסנן גאוסני עם רדיוס של 32 (BlurredFFT7.bmp) נותן לנו את התוצאה האידאלית ביותר עבור המשימה, וכעת נשמור אותו כעותק חדש בשם **Tim1LPFfft.bmp** ניתן להבחין שטשטוש זה משפיע בעוצמה גבוהה יותר מאשר טשטושים קודמים: ניתן לראות שהטקסט ברוב השורות נעשה קשה מאוד לקריאה, ובמיוחד בשורות התחתונות, זה בלתי אפשרי לקריאה.

מבחינת גרף ה Row Profile, ניכרת ירידה משמעותית בעוצמת השיאים, כלומר: הקווים הפכו לרכים, המעברים פחות חדים, והעקומה הפכה שטוחה יותר, מה שמעיד על אובדן של תדרים גבוהים ואיבוד פרטים עדינים. התוצאה מתאימה למשימה 4, מאחר שנבחר גודל מסנן שמטשטש את התמונה באופן כזה שלפחות 6 שורות הפכו לקריאה בקושי רב או לא קריאות כלל, כנדרש במשימה.

5. "RESTORE" ABOVE BLURRED IMAGES BY USING FFT

Fig 5.1-Tim1LPFfft.bmp



במשימה 5 אנו עוסקים בשחזור (unblurring) של תמונה שעברה טשטוש בתחום התדר, תוך ניסיון להשיב לה את הפרטים שאבדו. התהליך מבוצע באמצעות יישום של פילטר הופכי (inverse filter) בתדר, אשר מנסה לפצות על הטשטוש שהתקבל במסנן גאוס. בשלב זה נבחנים רדיוסים שונים של הפילטר ההפוך על מנת למצוא את הערך האופטימלי שמאפשר שיפור מרבי באיכות התמונה, כך שהטקסט יהפוך שוב לברור וחד ככל האפשר, מבלי להכניס רעשים מיותרים או עיוותים.

5. "restore" above blurred images by using FFT-continuation

Fig 5.2-gaussFilter20Reversed.bmp

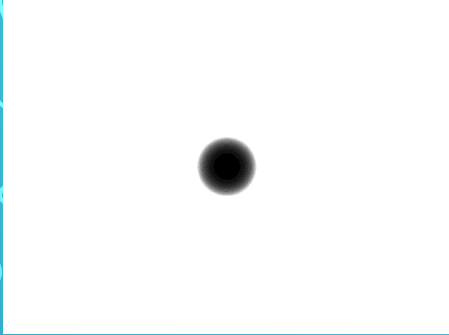


Fig 5.3-unBlur1.bmp



ניסינו לשחזר את התמונה המטושטשת על ידי שימוש בפילטר הפוך בתחום התדר, עם רדיוס 20.

התוצאה שהתקבלה תמונה אפורה וחלשה, התמונה עצמה אפילו יותר מטושטשת מהתמונה המקורית.

Fig 5.4-gaussFilter25Reversed.bmp

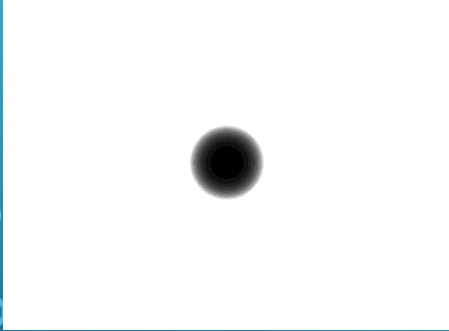


Fig 5.5-unBlur2.bmp



ניסינו לשחזר את התמונה המטושטשת על ידי שימוש בפילטר הפוך בתחום התדר, עם רדיוס 25.

התוצאה שהתקבלה שהתמונה עדיין נשארה אפורה, והטקסט אינו מובן. לכן נמשיך להגדיל את הרדיוס עד שנשיג תוצאה טובה יותר.

Fig 5.6-gaussFilter30Reversed.bmp

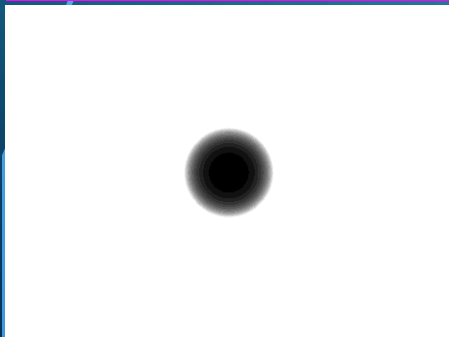


Fig 5.7-unBlur3.bmp



כעת הגדלנו את הרדיוס ל-30.

התוצאה שהתקבלה שהתמונה עדיין נשארה אפורה, אך רמת האפור ירדה בצורה נכבדה, והטקסט התחיל להיות מעט יותר חד וקריא, אך עדיין אינו מובן בשביל להיקרא בתור תמונת השחזור.

לכן נמשיך להגדיל את הרדיוס עד שנשיג תוצאה טובה יותר.

5. "restore" above blurred images by using FFT-continuation

Fig 5.8-gaussFilter35Reversed.bmp

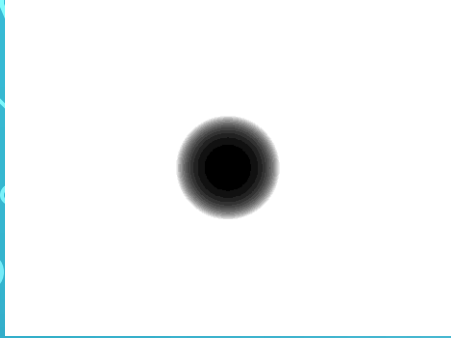


Fig 5.9-unBlur4.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

כעת הגדלנו את הרדיוס ל-35.

זוהי כמו התמונה הקודמת, רק שרמת האפור ירדה עוד יותר משמעותית.

לכן נמשיך להגדיל את הרדיוס עד שנשיג תוצאה טובה יותר.

Fig 5.10-gaussFilter40Reversed.bmp

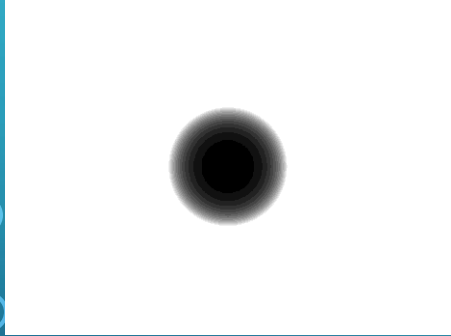


Fig 5.11-unBlur5.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

כעת הגדלנו את הרדיוס ל-40.

ניתן להבחין שהתוצאה שהתקבלה היא חדה מאוד: כל שורות הטקסט, כולל אלו שבחלק התחתון של התמונה.

בתמונה זו מתקבל איזון מוצלח בין שחזור חדות הטקסט לבין שמירה על מבנה התמונה המקורי

לכן נמשיך להגדיל את הרדיוס, במידה ונצליח להשיג תוצאה טובה יותר.

Fig 5.12-gaussFilter42Reversed.bmp

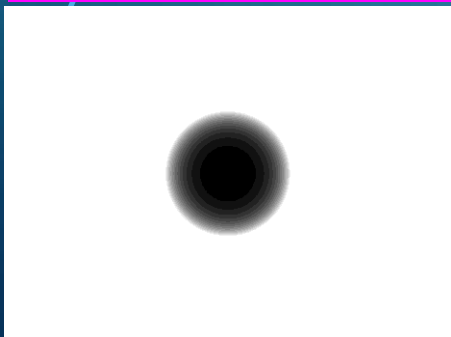


Fig 5.13-unBlur6.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

כעת הגדלנו את הרדיוס ל-42.

התמונה נראית מאוד מאוד זהה, למעט ה-3 שורות האחרונות שקריאות פחות, כלומר מטושטשות מעט ביחס לתמונה עם רדיוס 40.

לכן נמשיך להגדיל את הרדיוס, במידה ונצליח להשיג תוצאה טובה יותר.

5. "restore" above blurred images by using FFT-continuation

Fig 5.14-gaussFilter43Reversed.bmp

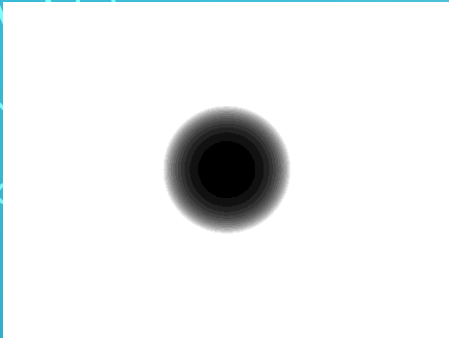


Fig 5.15-unBlur7.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

כעת הגדלנו את הרדיוס ל-42.
התמונה נראית מאוד מאוד זהה, למעט ה-3 שורות האחרונות שקריאות פחות, כלומר מטושטשות מעט ביחס לתמונה עם רדיוס 40.
לכן נמשיך להגדיל את הרדיוס, במידה ונצליח להשיג תוצאה טובה יותר.

Fig 5.16-gaussFilter48Reversed.bmp

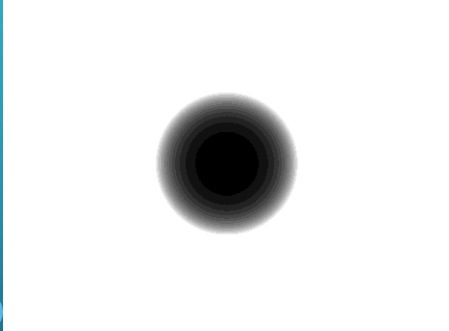


Fig 5.17-unBlur8.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

כעת הגדלנו את הרדיוס ל-48.
הטקסט הינו פחות חד ופחות מובן, ביחס לתמונה עם רדיוס 40.
לכן נמשיך להגדיל את הרדיוס, במידה ונצליח להשיג תוצאה טובה יותר.

Fig 5.18-gaussFilter50Reversed.bmp

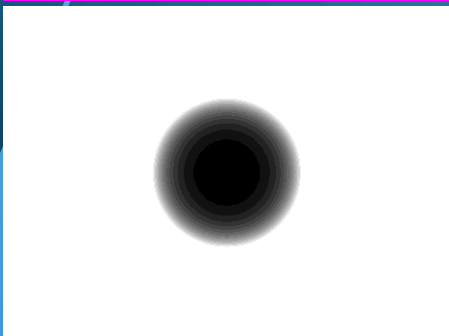


Fig 5.19-unBlur9.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

כעת הגדלנו את הרדיוס ל-50.
הטקסט הינו פחות חד ופחות מובן, ביחס לתמונה עם רדיוס 40.
לכן נמשיך להגדיל את הרדיוס, במידה ונצליח להשיג תוצאה טובה יותר.

5. "restore" above blurred images by using FFT-continuation

Fig 5.18-gaussFilter60Reversed.bmp

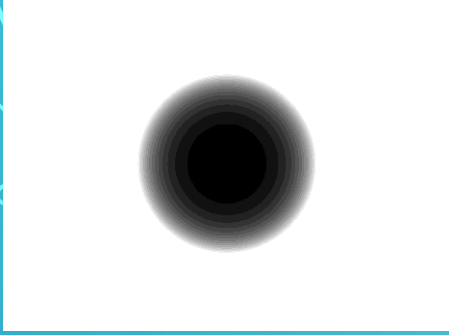


Fig 5.19-unBlur10.bmp



Fig 5.20-gaussFilter70Reversed.bmp

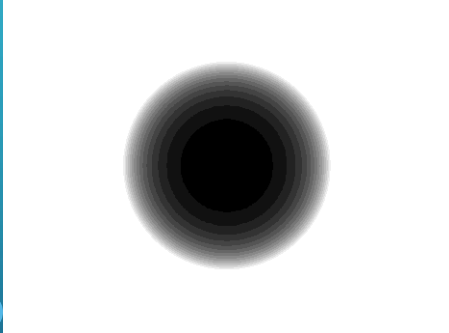


Fig 5.21-unBlur11.bmp



כעת הגדלנו את הרדיוס ל-60.

התמונה פחות טובה מהתמונה הקודמת.

נמשיך להגדיל את הרדיוס ונראה אם נשיג תוצאה איכותית יותר.

כעת הגדלנו את הרדיוס ל-70.

התמונה פחות טובה מהתמונה הקודמת, התחילה להיות מטושטשת מעט.

כפי שאנו רואים, רדיוס גדול יותר עושה עבודה טובה יותר, אבל בשלב מסוים, זה לא עושה שום שינוי ואפילו גורע מאיכות התמונה.

לכן נבחר ברדיוס 40 להיות המסנן שמחזר לנו את התמונה (שזוהי תמונה unBlur5.bmp)

5.Creating Tim1 RGfft.bmp

Fig 5.22-unBlur5.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

Row profile of fig 5.22

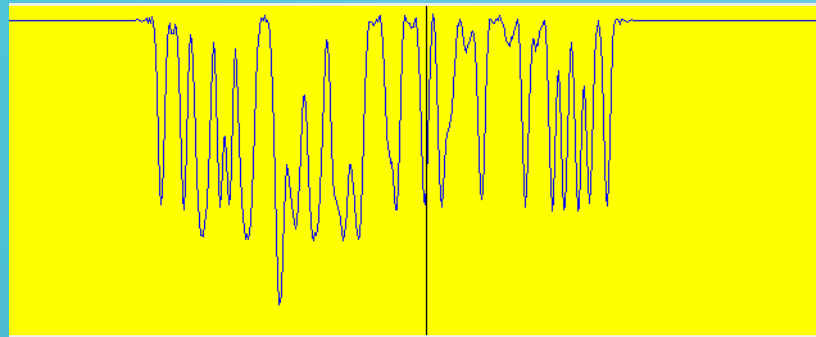


Fig 5.23-Tim1 RGfft.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

Row profile of fig 5.23

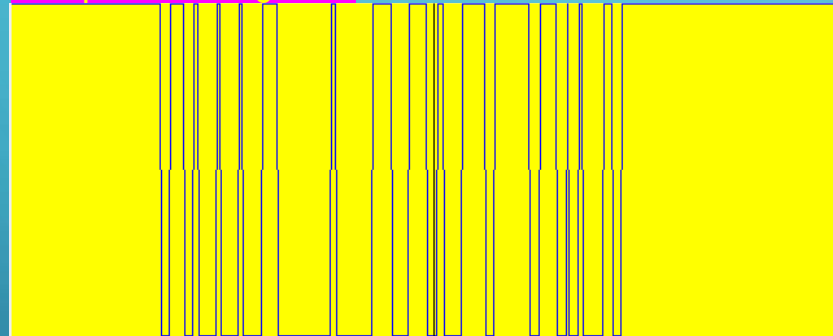
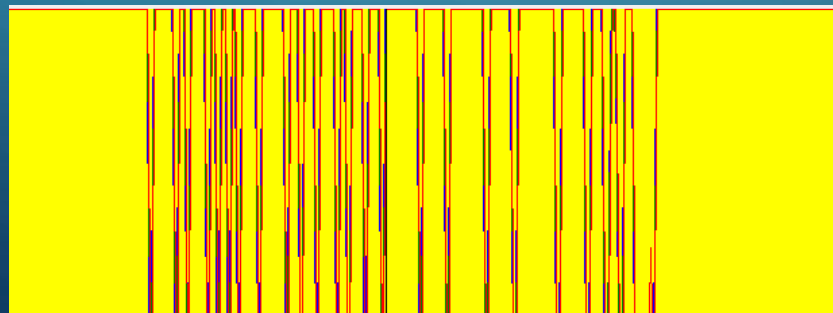


Fig 5.24-Tim1.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

Row profile of fig 5.25



לאחר השחזור בוצעה גם פעולה של נרמול מסוג MinMax, כאשר ערכים הגבוהים מ-200 הוגדרו כלבן, ויתר הערכים נפרשו על סקאלת האינטנסיביות בין שחור ללבן. התוצאה שהתקבלה דומה מאוד לתמונה המקורית המקורית Tim1.bmp

רוב הטקסט ברור וקריא, כולל השורות התחתונות, מה שמעיד על שחזור מוצלח של הפרטים החדים שאבדו בטשטוש.

5. Summary

Fig 5.25-Tim1.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

Row profile of fig 5.25

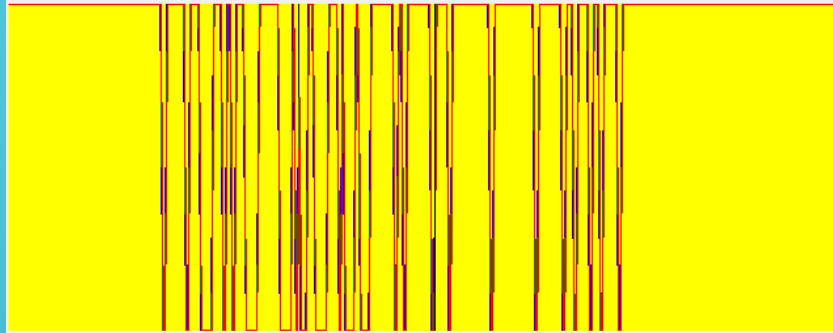


Fig 5.26-Tim1LPFfft.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

Row profile of fig 5.26

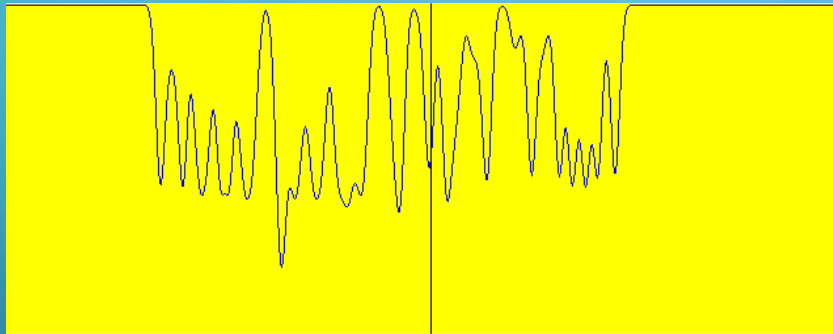
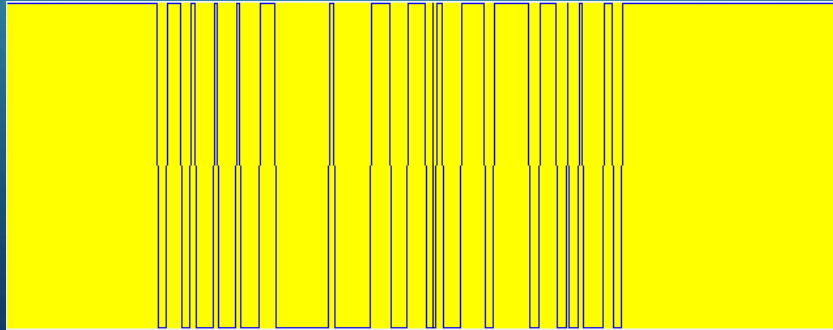


Fig 5.27-Tim1RGFfft.bmp

Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289
Hello from 1753-2289

Row profile of fig 5.27



התמונה המקורית (Tim1.bmp), הינה תמונה חדה וברורה, המציגה טקסטים במספר גדלים ובאופן ישר ומדויק. פרופיל השורה מציג קפיצות חדות ומוגדרות המייצגות את קצוות האותיות, מה שמעיד על חדות גבוהה וניגודיות טובה בין הטקסט לרקע.

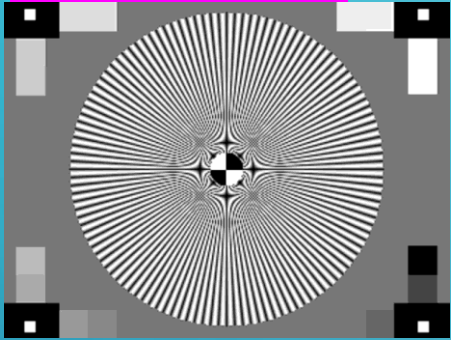
התמונה לאחר סינון Low-Pass (Tim1LPFfft.bmp) טושטשה בעזרת פילטר תחום תדר מסוג Low-Pass בתחום ה-FFT, מה שגרם לאובדן פרטים עדינים ולטשטוש כלל הטקסט. ניתן להבחין בטשטוש בקווי המתאר של האותיות, והפרופיל מציג מעבר רך בין רמות האפור, דבר המאפיין אובדן קונטרסט חד.

שחזור בעזרת פילטר הפוך (Tim1RGfft.bmp), מתבצע על ידי יישום פילטר הפוך מסוג

Inverse Gaussian בתחום התדר (FFT), במטרה להשיב את החדות שאבדה בטשטוש. התוצאה המתקבלת מראה שיפור חלקי בפרטים ובניגודיות, בעיקר בשורות העליונות של הטקסט (הגדולות יותר). בפרופיל ניתן להבחין בחזרה של קפיצות חדות בקצוות.

7. FILTRATION WITH FFT TO BLUR TIM2.BMP

Fig 7.1-Tim2.bmp



Row profile of fig 7.1

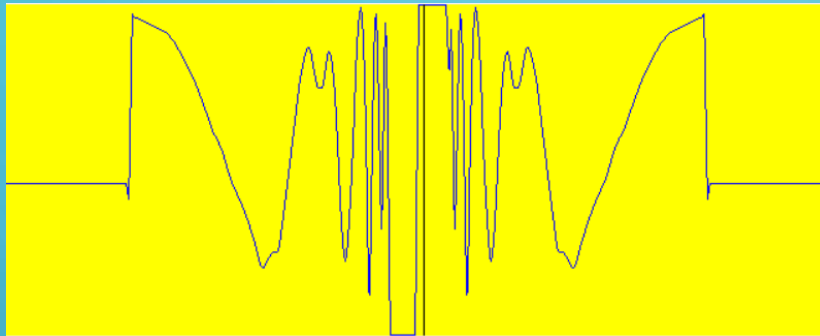


Fig 7.2-gaussFilter100.bmp

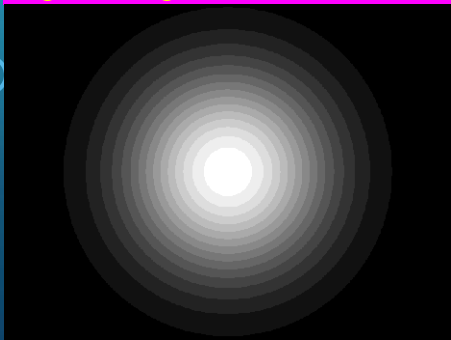
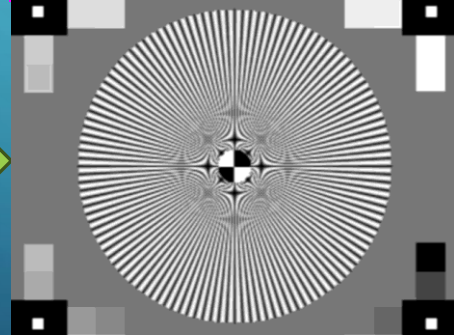
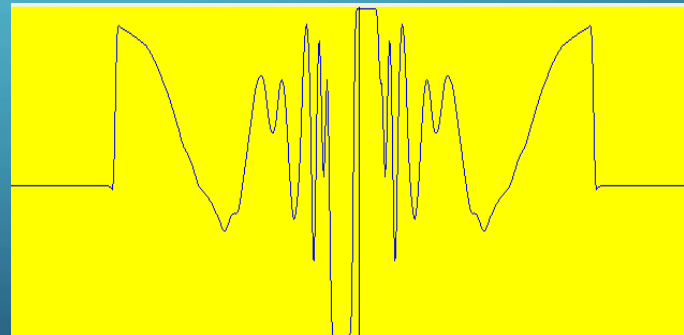


Fig 7.3- blurFFTtv1.bmp



Row profile of fig 7.3



התמונה המקורית עברה טשטוש בתחום התדר, על ידי מסנן גאوسی עם רדיוס של 100, מדובר בטשטוש עדין יחסית, ניתן להבחין כי רוב הפרטים בתמונה נשמרו, ובמיוחד האלמנטים המרכזיים בתמונה כמו קווי הרזולוציה נשארו חדים יחסית. עם זאת, ניתן להרגיש ירידה מסוימת בחדות הקווים באזורים הצפופים, בעיקר בכיוונים רדיאליים

בהתייחסות לתמונת הפרופיל, בתמונה המקורית ניתן לראות שבפרופיל המקורי קיימים שיאים חדים עם מעברים פתאומיים בין אזורים כהים ובהירים. לעומת זאת, לאחר ביצוע הטשטוש, המעברים בין השיאים הפכו מתונים ורכים יותר.

7.filtration with FFT to blur Tim2.bmp-continuation

Fig 7.4-gaussFilter80.bmp

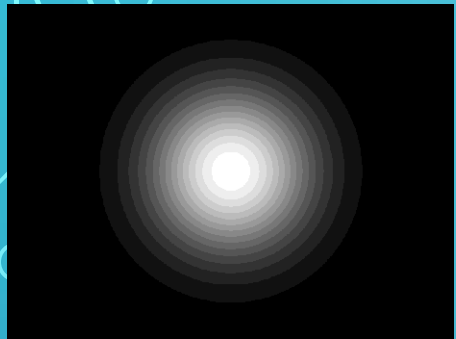
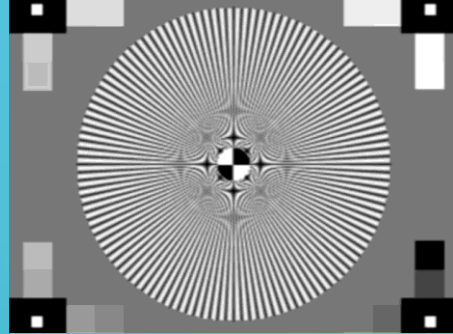
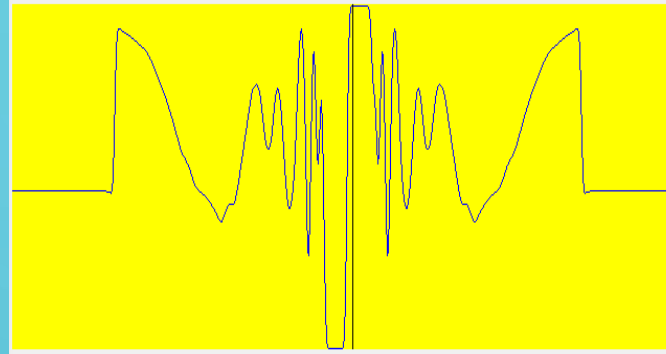


Fig 7.5-blurFFTtv2.bmp



Row profile of fig 7.5



כעת השתמשנו מסנן גאוס' עם רדיוס של 80, ניתן להבחין שיש פה טשטוש קל, אך רוב הפרטים הגיאומטריים בתמונה נשמרו, אך מורגש איבוד חדות מסוים, בעיקר בקווים דקים באזורים בעלי תדירות גבוהה. השיאים נעשו פחות חדים, הקונטרסט בין השיאים ירד והמעברים ביניהם הפכו רכים ומדורגים יותר

Fig 7.6-gaussFilter70.bmp

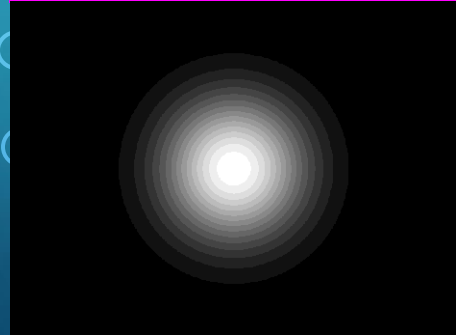
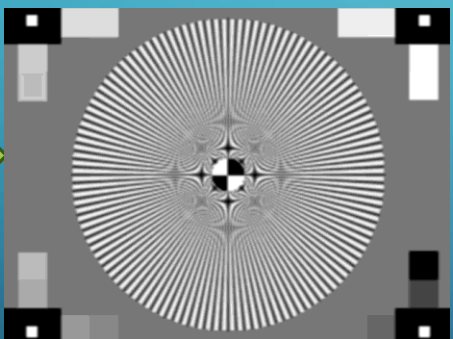
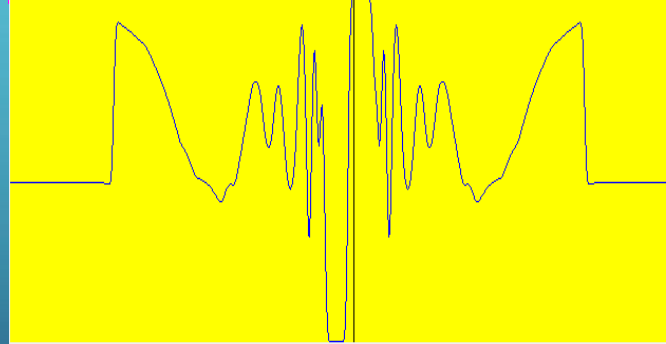


Fig 7.7-blurFFTtv3.bmp



Row profile of fig 7.7



כעת השתמשנו מסנן גאוס' עם רדיוס של 70, לא רואים הבדל בין התמונה הזו לתמונה הקודמת, מבחינת הטשטוש וגם מבחינת הפרופיל. לכן נלך ונקטין את הרדיוס ונבחן את ההשפעה.

7. filtration with FFT to blur Tim2.bmp-continuation

Fig 7.8-gaussFilter60.bmp

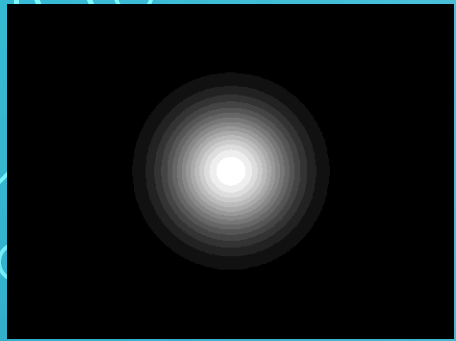
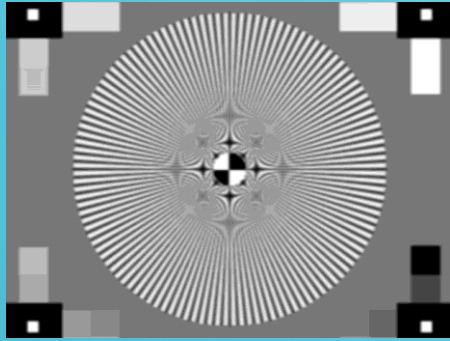
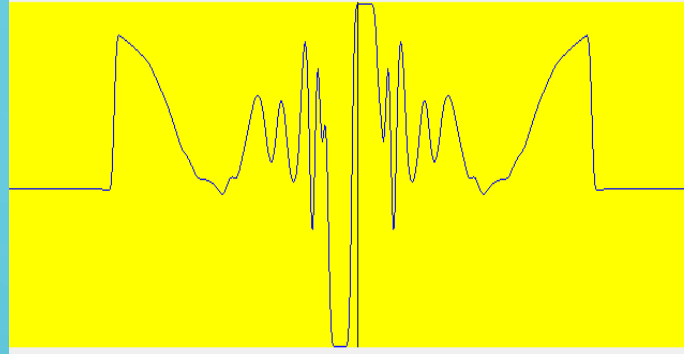


Fig 7.9-blurFFTtv4.bmp



Row profile of fig 7.9



כעת השתמשנו מסנן גאוס' עם רדיוס של 60, לא רואים הבדל בין התמונה הזו לתמונה הקודמת, מבחינת הטשטוש וגם מבחינת הפרופיל. לכן נלך ונקטין את הרדיוס ונבחן את ההשפעה.

Fig 7.10-gaussFilter50.bmp

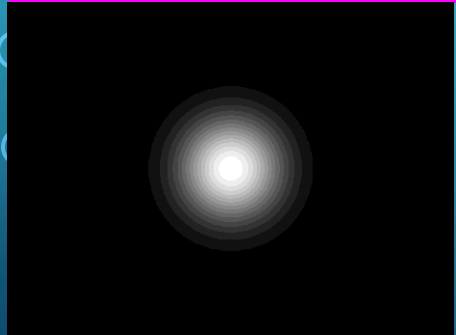
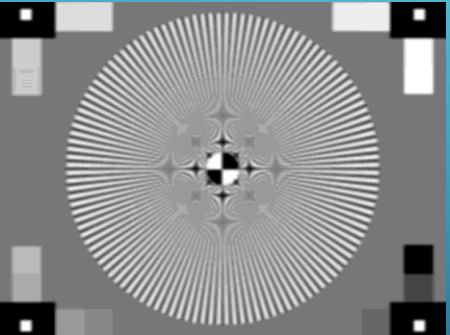
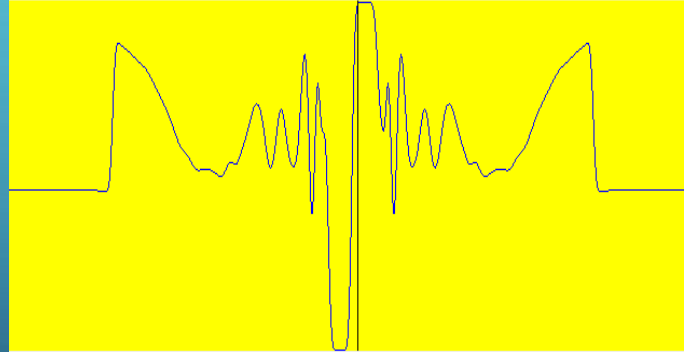


Fig 7.11-blurFFTtv5.bmp



Row profile of fig 7.11



כעת השתמשנו מסנן גאוס' עם רדיוס של 50, הטשטוש הפך למשמעותי הרבה יותר, אנחנו התקרבו בניסיון זה משמעותית לפתרון המשימה, בוצעה סינון אגרסיבי יותר של תדרים גבוהים. בתמונה עצמה, פרטים קטנים וחזקים כמו קווים דקים איבדו מחדותם. הפרופיל נעשה חלק יותר בצורה. ניתן להבחין בפרטים עדינים מסוימים, כלומר, הטשטוש אינו מספק לחלוטין.

7. filtration with FFT to blur Tim2.bmp-continuation

Fig 7.12-gaussFilter40.bmp

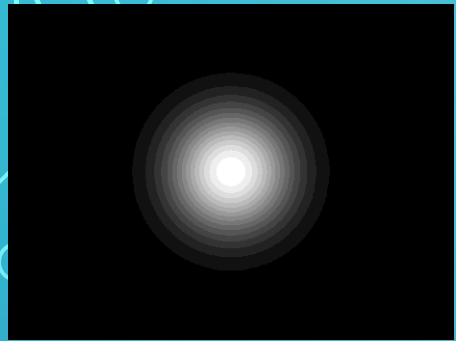
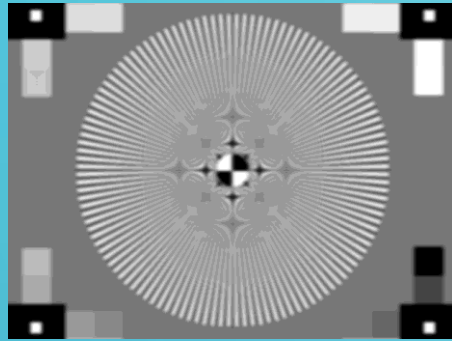
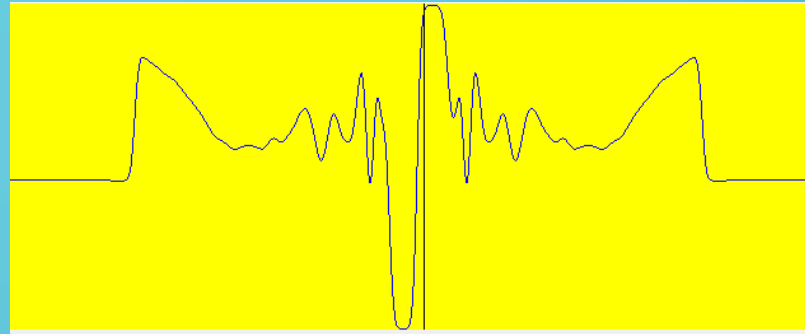


Fig 7.13-Tim2LPFfft.bmp



Row profile of fig 7.13



כעת השתמשנו מסנן גאוסני עם רדיוס של 40, ניתן להבחין שטשטוש זה משפיע בעוצמה גבוהה יותר מאשר טשטושים קודמים: התוצאה מציגה טשטוש אפקטיבי ומשמעותי מאוד, כמעט כל הקווים הדקים והפרטים הגאומטריים הקטנים נעלמו.

מבחינת גרף ה Row Profile, ניכרת ירידה משמעותית בעוצמת השיאים, במיוחד באזור המרכזי. השיאים הגבוהים נחתכו כמעט לגמרי, והמעברים הפכו חלקים ורחבים יותר, מה שמעיד על סינון מוצלח של תדרים גבוהים.

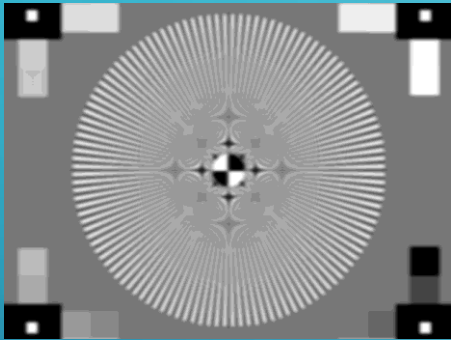
התוצאה מתאימה למשימה 7, מאחר שנבחר גודל מסנן שמטשטש את התמונה באופן כך שעונה על דרישת המשימה.

בנוסף, הושגה דיכוי יעיל של פרטים חדים בתמונה.

ניכר כי רדיוס 40 סיפק את רמת הטשטוש הנדרשת, ומחק כמעט כל מאפיין חד בתמונה המקורית. מבחינת הקריטריונים של המשימה, מדובר בתוצאה שעונה על דרישת המשימה.

7. "RESTORE" ABOVE BLURRED IMAGES BY USING FFT

Fig 7.14-Tim2LPFfft.bmp



במשימה אנו עוסקים בשחזור (unblurring) של תמונה שעברה טשטוש בתחום התדר, תוך ניסיון להשיב לה את הפרטים שאבדו. התהליך מבוצע באמצעות יישום של פילטר הופכי (inverse filter) בתדר, אשר מנסה לפצות על הטשטוש שהתקבל במסנן גאומטרי. בשלב זה נבחרים רדיוסים שונים של הפילטר ההפוך על מנת למצוא את הערך האופטימלי שמאפשר שיפור מרבי באיכות התמונה – כך שהטקסט יהפוך שוב לברור וחד ככל האפשר, מבלי להכניס רעשים מיותרים או עיוותים.

7. "restore" above blurred images by using FFT-continuation

Fig 7.15 gaussFilter20Reversed.bmp

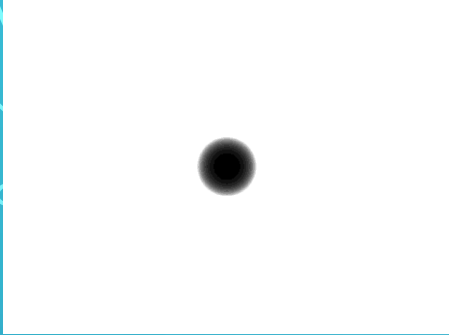
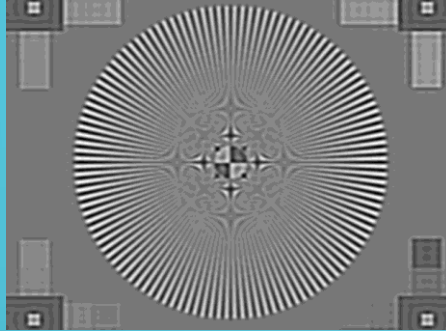


Fig 7.16-unblurFFTtv1.bmp



ניסינו לשחזר את התמונה המטושטשת על ידי שימוש בפילטר הפוך בתחום התדר, עם רדיוס 20.

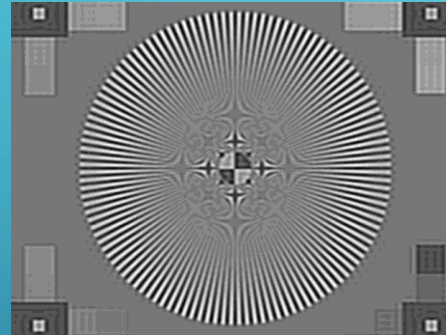
ניתן להבחין בשיפור קל בלבד בקונטרסט, בעיקר באזורים בעלי מעברים חדים, אך הפרטים הדקים והעדינים, כגון גבולות גאומטריים, נותרו מטושטשים.

לכן נמשיך להגדיל את הרדיוס עד שנשיג תוצאה טובה יותר.

Fig 7.17-gaussFilter25Reversed.bmp



Fig 7.18-unblurFFTtv2.bmp



ניסינו לשחזר את התמונה המטושטשת על ידי שימוש בפילטר הפוך בתחום התדר, עם רדיוס 25.

ישנו שיפור קל בתמונה של הקונטרסט, לעומת התמונה הקודמת, למרות השיפור, השחזור עדיין אינו מספק את רמת החדות הרצויה.

לכן נמשיך להגדיל את הרדיוס עד שנשיג תוצאה טובה יותר.

Fig 7.19-gaussFilter30Reversed.bmp

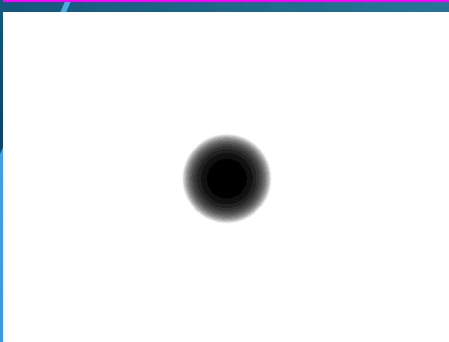
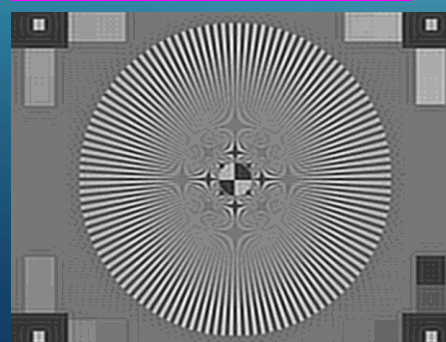


Fig 7.20-unblurFFTtv3.bmp



כעת הגדלנו את הרדיוס ל-30.

התוצאה שהתקבלה שהתמונה טיפה פחות אפורה ויותר חדה בצורה משמעותית, למרות זאת התמונה עדיין בעלת טשטוש רב.

לכן נמשיך להגדיל את הרדיוס עד שנשיג תוצאה טובה יותר.

7. "restore" above blurred images by using FFT-continuation

Fig 7.21-gaussFilter35Reversed.bmp

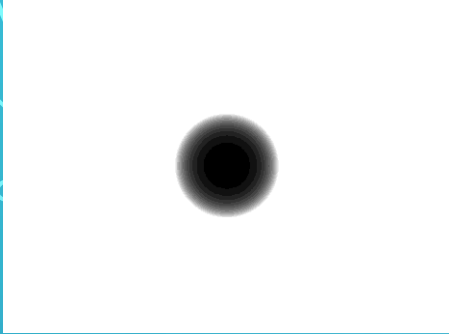
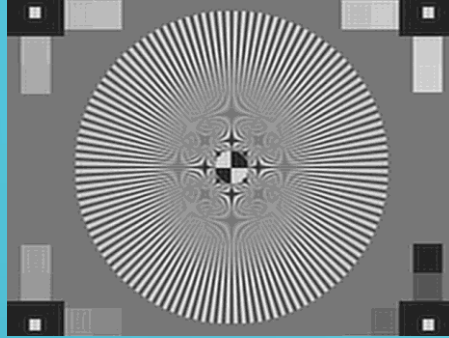


Fig 7.22-unblurFFTtv4.bmp



כעת הגדלנו את הרדיוס ל-35.

ברדיוס זה השחזור נעשה מורגש הרבה יותר. ניתן לראות שיפור ניכר בפרטים הקטנים של התמונה, גבולות ברורים יותר מתחילים להופיע, והקונטרסט הכללי ממשיך להשתפר. לכן נמשיך להגדיל את הרדיוס עד שנשיג תוצאה טובה יותר.

Fig 7.23-gaussFilter40Reversed.bmp

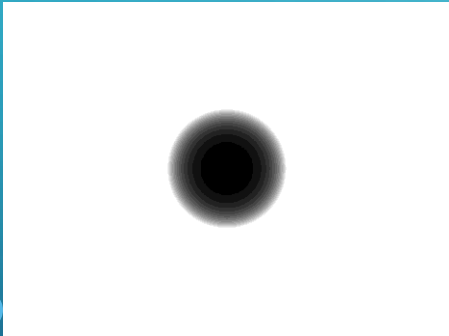
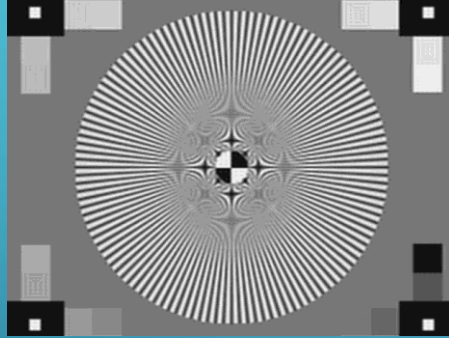


Fig 7.24-unblurFFTtv5.bmp



כעת הגדלנו את הרדיוס ל-40.

ישנו שיפור משמעותי בתמונה מבחינת החדות. הטשטוש, אך עדיין קיימים אזורים מעורפלים, מה שמעיד על שיפור חלקי בלבד. לכן נמשיך להגדיל את הרדיוס עד שנשיג תוצאה טובה יותר.

Fig 7.25-gaussFilter42Reversed.bmp

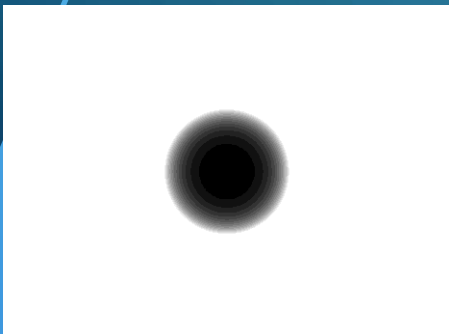
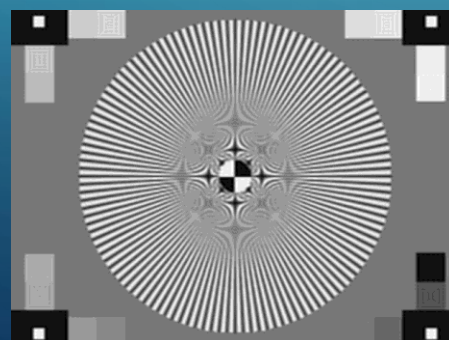


Fig 7.26-unblurFFTtv6.bmp



כעת הגדלנו את הרדיוס ל-42.

אנו מקבלים שיחזור מוצלח מאוד של התמונה, רוב הפרטים שנמחקו בטשטוש חזרו, קווים דקים, גבולות גאומטריים ומעברים חדים הפכו ברורים וחדים יותר. בהשוואה לניסיונות הקודמים, זהו השיחזור האיכותי ביותר.

7. "restore" above blurred images by using FFT-continuation

Fig 7.27-gaussFilter45Reversed.bmp

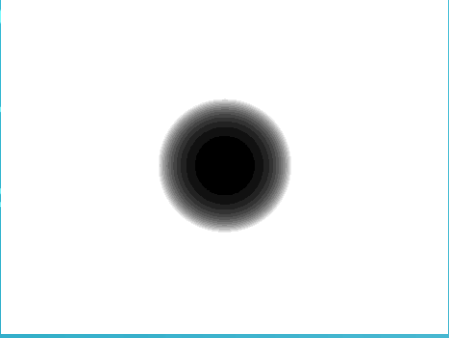
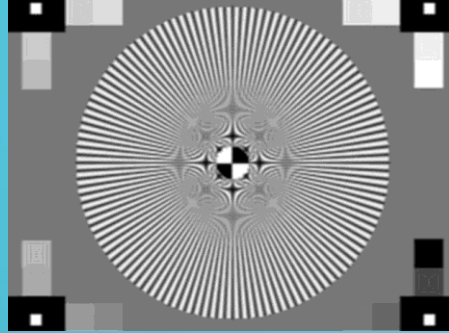


Fig 7.28-unblurFFTtv7.bmp



כעת הגדלנו את הרדיוס ל-45.

התמונה הזו מאוד מאוד דומה לתמונה בעלת רדיוס 42, אך הקונטרסט בין האזורים בתמונה מוגזם מעט.

Fig 7.29-gaussFilter48Reversed.bmp

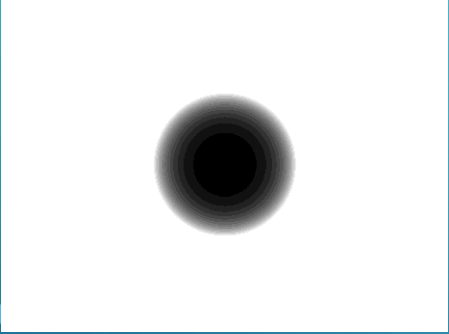
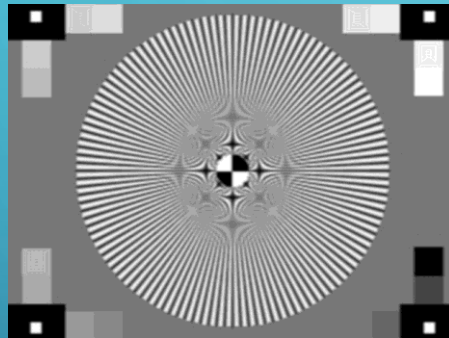


Fig 7.30-unblurFFTtv8.bmp



כעת הגדלנו את הרדיוס ל-48.

התמונה הזו מאוד מאוד דומה לתמונה בעלת רדיוס 42, אך הקונטרסט בין האזורים בתמונה מוגזם עוד יותר מהתמונה הקודמת (רדיוס 45), נוצרות תופעות לוואי כמו הדגשה מוגזמת של גבולות ומריחות מקומיות.

Fig 7.31-gaussFilter50Reversed.bmp

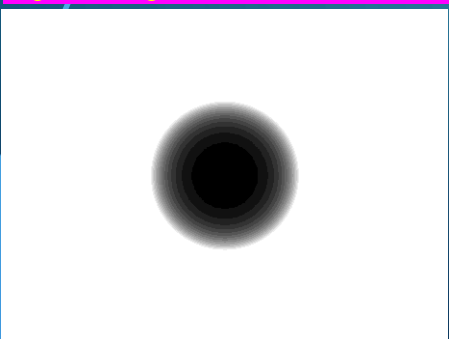
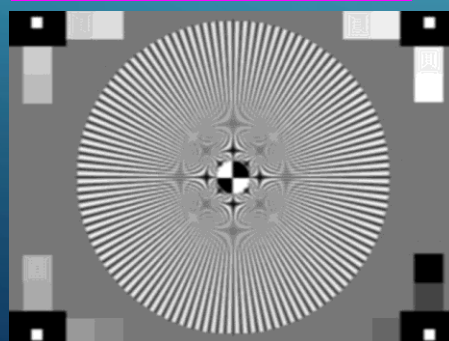


Fig 7.32-unblurFFTtv9.bmp



כעת הגדלנו את הרדיוס ל-50.

התמונה פחות טובה מהתמונה הקודמת.

7. "restore" above blurred images by using FFT-continuation

Fig 7.33-gaussFilter60Reversed.bmp

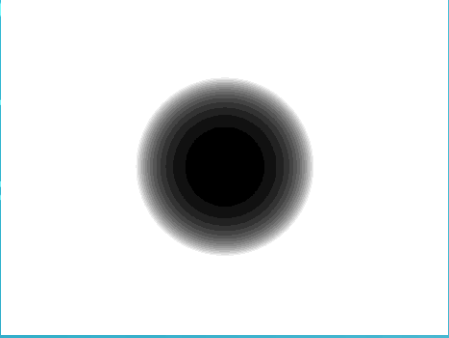
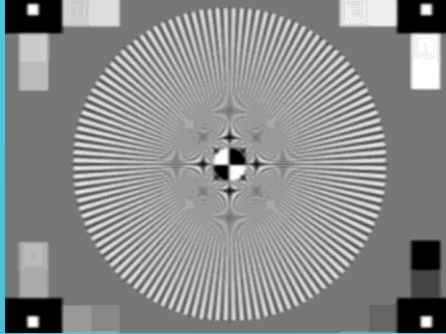


Fig 7.34-unblurFFTtv10.bmp



כעת הגדלנו את הרדיוס ל-60.

מתקבלת תמונה עם חידוד מוגזם ועיוותים בולטים.
במקום שיפור איכות, השחזור מדגיש רעש וגורם להופעת קווים לא טבעיים באזורים שהיו אמורים להישאר חלקים.

Fig 7.35-gaussFilter70Reversed.bmp

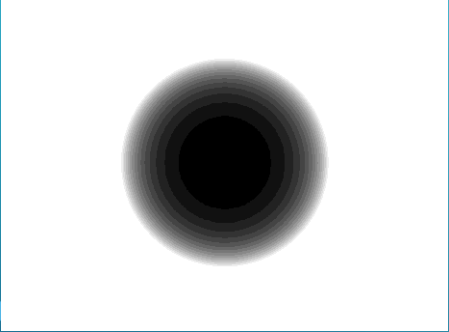
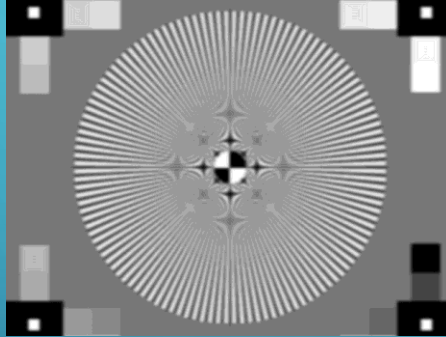


Fig 7.36-unblurFFTtv11.bmp



כעת הגדלנו את הרדיוס ל-70.

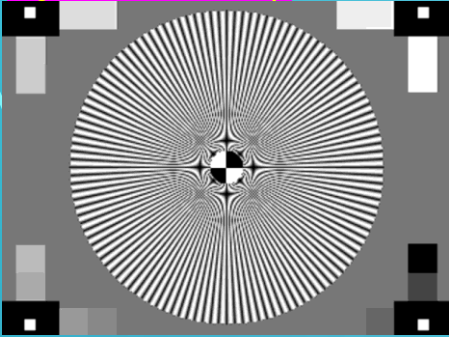
התמונה פחות טובה מהתמונה הקודמת, התחילה להיות מטושטשת מעט.

כפי שאנו רואים, רדיוס גדול יותר עושה עבודה טובה יותר, אבל בשלב מסוים, זה לא עושה שום שינוי ואפילו גורע מאיכות התמונה.

לכן נבחר ברדיוס 42 להיות המסנן שמחזר לנו את התמונה (שזוהי תמונה unblurFFTtv6.bmp)

7. Summary

Fig 7.37-Tim2.bmp



Row profile of fig 7.37

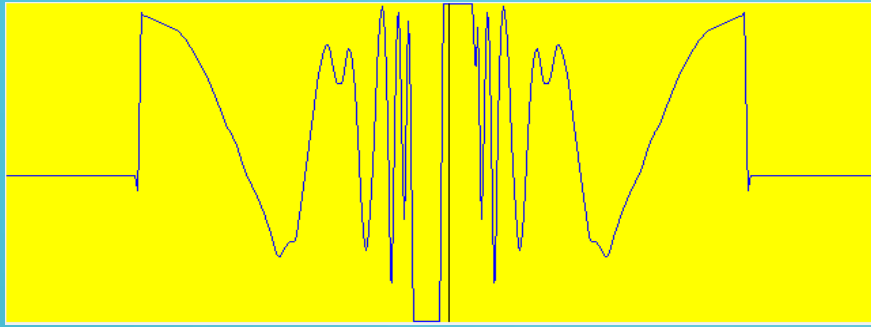
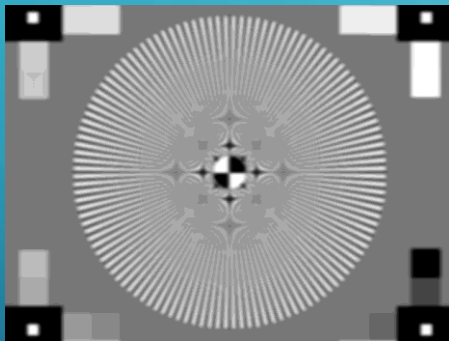


Fig 7.38-Tim2LPFfft-.bmp



Row profile of fig 7.38

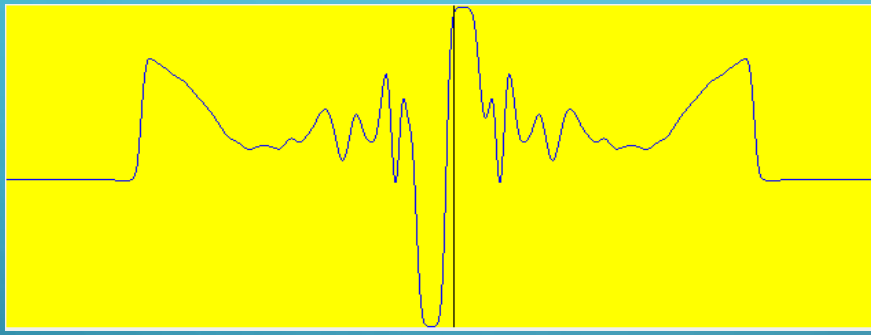
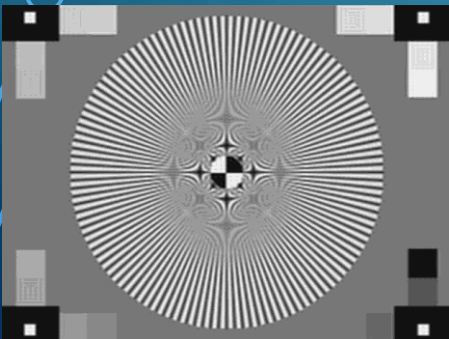
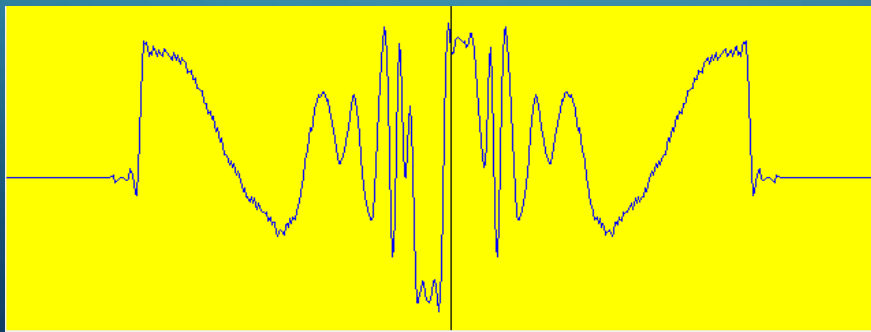


Fig 7.39-Tim2RGfft.bmp



Row profile of fig 7.39



התמונה המקורית (Tim2.bmp), התמונה חדה וברורה, כוללת פרטים עדינים כמו גבולות גאומטריים דקים וטקסט. פרופיל השורה מציג שיאים חדים המעידים על מעברים חדים בתמונה.

התמונה לאחר טשטוש (Tim2LPFfft.bmp), סוננה בעזרת מסנן Low-Pass בתחום התדר, מה שגרם לטשטוש כללי של התמונה. המעברים החדים נעלמו, הקונטרסט ירד, והפרטים הדקים הוחלקו. זה ניכר גם בפרופיל השורה, השיאים החדים התמתנו, והגרף הפך רחב וחלק יותר.

שחזור בעזרת פילטר הפוך (Tim2RGfft.bmp), ניסינו לשחזר את התמונה בעזרת פילטר הפוך מסוג Inverse Gaussian בתדר, עם רדיוס אופטימלי (שיצא לנו : 42). התמונה שופרה בצורה משמעותית ביחס לגרסה המטושטשת, קונטרסט גבוה יותר, שיפור בגבולות ובפרטים חדים.

בפרופיל השורה ניתן לראות חזרה חלקית של השיאים, אך עם רעש קל.

Code of “minmax” function

```
28 void minmax(int ThreshHold, string filename, c2dFloatGrayImage& src, c2dByteGrayImage& des)
29 {
30     int numPixels = src.NumberOfPixels();
31     tFloat* pSrc = src.PointerToPixel(); // Pointer to source image
32     tByte* pDes = des.PointerToPixel(); // Pointer to destination image
33
34     // Find min and max
35     tFloat minVal = pSrc[0], maxVal = pSrc[0];
36     for (int i = 1; i < numPixels; i++) {
37         if (pSrc[i] < minVal) minVal = pSrc[i];
38         if (pSrc[i] > maxVal) maxVal = pSrc[i];
39     }
40
41     if (minVal == maxVal) return;
42
43     double scale = 255.0 / (maxVal - minVal);
44
45     for (int i = 0; i < numPixels; i++) {
46         tByte newVal = (tByte)((pSrc[i] - minVal) * scale);
47         pDes[i] = (newVal > ThreshHold) ? 255 : 0;
48     }
49
50     // Save image
51     string t_filename = filename + ".bmp";
52     des.SaveToGrayBmpFile(&t_filename[0]);
53 }
```

הפונקציה minmax מקבלת תמונת מקור בגווי אפור בפורמט float, מחשבת את הערכים המינימלי והמקסימלי בתמונה, ומבצעת נרמול לטווח של 0 עד 255. לאחר מכן, היא מבצעת סף (Threshold), כל ערך הגדול מהסף שניתן יהפוך ל-255 (לבן), וכל ערך נמוך או שווה יהפוך ל-0 (שחור). לבסוף, היא שומרת את התוצאה בפורמט BMP בשם שנקבע. מטרת הפונקציה היא להפוך תוצאות של עיבוד תמונה (כגון קונבולוציה או סינון) לתמונה בינארית הניתנת להצגה והשוואה.

Code of “DoGaussianFiltration” function

```
56 void DoGaussianFiltration(c2dByteGrayImage& source, double* filter)
57 {
58     // Get the number of rows and columns in the source image
59     int numRows = source.NumberOfRows();
60     int numColumns = source.NumberOfColumns();
61
62     // Create a temporary destination image with the same dimensions as the source image
63     c2dByteGrayImage temp_dest(numRows, numColumns);
64     double summa;
65
66     // Apply filtration in the x direction (horizontal convolution)
67     for (int row = 0; row < numRows; row++)
68     {
69         for (int column = filterHalfSize; column < numColumns - filterHalfSize; column++)
70         {
71             summa = 0;
72             // Convolution operation along the x direction
73             for (int x = -filterHalfSize; x <= filterHalfSize; x++)
74             {
75                 summa = summa + filter[filterHalfSize + x] * (double)source.GetPixelValue(row, column + x);
76             } // end of x convolution
77
78             // Clamp the result to the range [0, 255] to prevent overflow or underflow
79             if (summa < 0)
80                 summa = 0;
81             if (summa > 255)
82                 summa = 255;
83
84             // Store the result in the temporary destination image (rounding to nearest byte value)
85             temp_dest.SetPixelValue(row, column, (tByte)(summa + 0.5));
86         } // end of column loop
87     } // end of row loop
88
89     // Clear the source image to prepare for vertical (y-direction) filtering
90     source.Clear();
91
92     // Apply filtration in the y direction (vertical convolution)
93     for (int row = filterHalfSize; row < numRows - filterHalfSize; row++)
94     {
95         for (int column = 0; column < numColumns; column++)
96         {
97             summa = 0;
98             // Convolution operation along the y direction
99             for (int y = -filterHalfSize; y <= filterHalfSize; y++)
100             {
101                 summa += filter[filterHalfSize + y] * (double)temp_dest.GetPixelValue(row + y, column);
102             } // end of y convolution
103
104             // Clamp the result to the range [0, 255] to prevent overflow or underflow
105             if (summa < 0)
106                 summa = 0;
107             if (summa > 255)
108                 summa = 255;
109
110             // Store the result back in the source image (rounding to nearest byte value)
111             source.SetPixelValue(row, column, (tByte)(summa + 0.5));
112         } // end of column loop
113     } // end of row loop
114 }
```

הפונקציה DoGaussianFiltration מבצעת טשטוש לתמונה בגווני אפור באמצעות סינון גאוסי חד-ממדי בשני שלבים.

בשלב הראשון היא מבצעת קונבולוציה אופקית (לאורך שורות התמונה) עם פילטר גאוסי נתון, ושומרת את התוצאה בתמונה זמנית.

לאחר מכן, היא מבצעת קונבולוציה אנכית (לאורך עמודות) על אותה תוצאה זמנית, ומשלימה בכך סינון דו-ממדי אפקטיבי. בכל שלב מבוצעת הגבלה של ערכי הפיקסלים לטווח התקני 0 עד 255, והתוצאה הסופית היא תמונה מטושטשת באופן חלק ועדין.

Code of “DoFiltrationByConvolutionMINMAX” function

```
116
117 // Applies a convolution filter to the source image and stores the result in the destination image.
118 void DoFiltrationByConvolutionMINMAX(string filename, c2dByteGrayImage& source, c2dByteGrayImage& dest, double filter[][3], double factor, int threshold)
119 {
120     int filterHalfSize = 1;
121     int numRows = source.NumberOfRows();
122     int numCols = source.NumberOfColumns();
123     int numPixels = source.NumberOfPixels();
124
125     // Clear destinations
126     dest.Clear();
127
128     // Allocate temporary float image
129     c2dFloatGrayImage temp(numRows, numCols);
130
131     // Access raw data with pointers
132     tByte* pSrc = source.PointerToPixel();
133     tFloat* pTemp = temp.PointerToPixel();
134
135     // Convolution loop using pointer access only
136     for (int row = filterHalfSize; row < numRows - filterHalfSize; row++) {
137         for (int col = filterHalfSize; col < numCols - filterHalfSize; col++) {
138             double summa = 0.0;
139
140             for (int dy = -filterHalfSize; dy <= filterHalfSize; dy++) {
141                 for (int dx = -filterHalfSize; dx <= filterHalfSize; dx++) {
142                     int neighborRow = row + dy;
143                     int neighborCol = col + dx;
144
145                     double pixelVal = (double)pSrc[neighborRow * numCols + neighborCol];
146                     double filterVal = filter[dy + filterHalfSize][dx + filterHalfSize];
147
148                     summa += pixelVal * filterVal;
149                 }
150             }
151
152             summa *= factor;
153             pTemp[row * numCols + col] = (tFloat)summa;
154         }
155     }
156
157     // Apply the min-max normalization with thresholding and save the result
158     minmax(threshold, filename, temp, dest);
159 }
```

הפונקציה

DoFiltrationByConvolutionMINMAX מבצעת קונבולוציה של תמונה בגווי אפור עם מסנן 3X3, תוך שימוש בגישה דרך מצביעים לשיפור יעילות. תחילה היא מקצה תמונת ביניים מסוג float לאחסון תוצאות החישוב המדויקות. לאחר מכן, על כל פיקסל בתמונה (למעט הגבולות), היא מחשבת סכום משוקלל של פיקסלים סמוכים על פי ערכי המסנן הנתון ומכפילה בפקטור נורמליזציה. בסוף, היא מפעילה על התוצאה נרמול לפי מינימום ומקסימום עם סף (threshold) באמצעות הפונקציה minmax, וכותבת את התמונה הבינארית הסופית לקובץ BMP בשם שניתן. התוצאה היא תמונה מסוננת שעברה גם סף, ומשמשת בדרכ כלל להדגשת פרטים או שיחזור תדרים גבוהים.

Code of “DoFiltrationByConvolution” function

```
161 // Applies a convolution filter to the source image and stores the result in the destination image.
162 // Applies a 3x3 convolution filter on the source image and stores the result in the destination image.
163 void DoFiltrationByConvolution(c2dByteGrayImage& source, c2dByteGrayImage& dest, double filter[][3], double factor)
164 {
165     int filterHalfSize = 1; // for 3x3 filter
166     dest.Clear(); // reset destination image
167     int numRows = source.NumberOfRows();
168     int numCols = source.NumberOfColumns();
169
170     tByte* pSrc = source.PointerToPixel();
171     tByte* pDst = dest.PointerToPixel();
172
173     for (int row = filterHalfSize; row < numRows - filterHalfSize; row++) {
174         for (int col = filterHalfSize; col < numCols - filterHalfSize; col++) {
175             double summa = 0.0;
176
177             for (int dy = -filterHalfSize; dy <= filterHalfSize; dy++) {
178                 for (int dx = -filterHalfSize; dx <= filterHalfSize; dx++) {
179                     int neighborRow = row + dy;
180                     int neighborCol = col + dx;
181                     double pixelValue = (double)pSrc[neighborRow * numCols + neighborCol];
182                     double filterValue = filter[dy + filterHalfSize][dx + filterHalfSize];
183                     summa += pixelValue * filterValue;
184                 }
185             }
186
187             summa *= factor;
188             if (summa < 0) summa = 0;
189             if (summa > 255) summa = 255;
190
191             pDst[row * numCols + col] = (tByte)(summa + 0.5);
192         }
193     }
194 }
```

הפונקציה DoFiltrationByConvolution מבצעת קונבולוציה של תמונה בגווי אפור עם פילטר בגודל 3X3, ומאחסנת את התוצאה בתמונה חדשה. כל פיקסל בתמונה עובר חישוב של סכום משוקלל לפי ערכי המסנן (filter) באזור השכנים שלו, כשהתוצאה מוכפלת בפקטור נורמליזציה (factor). לאחר מכן, הפלט מוגבל לטווח תקי של ערכי פיקסלים (0 עד 255), מעוגל ומועתק לתמונת היעד. הפונקציה משתמשת בגישה יעילה דרך מצביעים לגישה ישירה למידע הפיקסלים. מטרת הפונקציה היא לבצע סינון מסוג "מרחבי" (spatial filtering) לצורך חידוד, טשטוש או הדגשת קצוות, תלוי במסנן שניתן.

Code of “DoFiltrationByConvolution5MINMAX” function

```
197 // Applies a 5x5 convolution filter and then normalizes the result using min-max normalization.
198 void DoFiltrationByConvolution5MINMAX(string filename, c2dByteGrayImage& source, c2dByteGrayImage& dest, double filter[][5], double factor, int threshold)
199 {
200     int filterHalfSize = 2; // 5x5 filter -> half = 2
201     int numRows = source.NumberOfRows();
202     int numCols = source.NumberOfColumns();
203     int numPixels = source.NumberOfPixels();
204
205     // Clear destination image
206     dest.Clear();
207
208     // Temporary float image to store filtered result
209     c2dFloatGrayImage temp(numRows, numCols);
210
211     // Get raw data pointers
212     tByte* pSrc = source.PointerToPixel();
213     tFloat* pTemp = temp.PointerToPixel();
214
215     // Apply convolution using pointer access
216     for (int row = filterHalfSize; row < numRows - filterHalfSize; row++) {
217         for (int col = filterHalfSize; col < numCols - filterHalfSize; col++) {
218             double summa = 0.0;
219
220             for (int dy = -filterHalfSize; dy <= filterHalfSize; dy++) {
221                 for (int dx = -filterHalfSize; dx <= filterHalfSize; dx++) {
222                     int neighborRow = row + dy;
223                     int neighborCol = col + dx;
224
225                     double pixelVal = (double)pSrc[neighborRow * numCols + neighborCol];
226                     double filterVal = filter[dy + filterHalfSize][dx + filterHalfSize];
227
228                     summa += pixelVal * filterVal;
229                 }
230             }
231
232             summa *= factor;
233             pTemp[row * numCols + col] = (tFloat)summa;
234         }
235     }
236
237     // Apply min-max normalization + thresholding
238     minmax(threshold, filename, temp, dest);
239 }
```

הפונקציה

DoFiltrationByConvolution5MINMAX מבצעת

קונבולוציה של תמונה בגוויי אפור עם פילטר

בגודל 5X5, ומבצעת נרמול מסוג Min-Max עם

סף. הפונקציה יוצרת תמונת ביניים מסוג float

שבה נשמרות תוצאות הקונבולוציה המדויקות,

המחושבות על ידי סכום משוקלל של ערכי

הפיקסלים הסמוכים, מוכפלים במסנן הנתון

ובפקטור נורמליזציה. לאחר החישוב, מבוצעת

נרמול של הערכים לטווח 255–0 באמצעות

הפונקציה minmax, יחד עם סף (threshold)

לקביעת אם כל פיקסל יהיה לבן או שחור.

הפונקציה מתאימה להדגשת תדרים גבוהים

(חידוד) או לזיהוי פרטים עדינים בהתאם למסנן

שנבחר.

Code of “DoFiltrationByConvolution5” function

```
240
241 // Applies a 5x5 convolution filter and clamps the result between [0, 255].
242 void DoFiltrationByConvolution5(c2dByteGrayImage& source, c2dByteGrayImage& dest, double filter[][5], double factor)
243 {
244     int filterHalfSize = 2; // for 5x5 filter
245     int numRows = source.NumberOfRows();
246     int numCols = source.NumberOfColumns();
247
248     // Clear destination image
249     dest.Clear();
250
251     // Get raw data pointers
252     tByte* pSrc = source.PointerToPixel();
253     tByte* pDst = dest.PointerToPixel();
254
255     // Apply convolution using pointer access
256     for (int row = filterHalfSize; row < numRows - filterHalfSize; row++) {
257         for (int col = filterHalfSize; col < numCols - filterHalfSize; col++) {
258             double summa = 0.0;
259
260             for (int dy = -filterHalfSize; dy <= filterHalfSize; dy++) {
261                 for (int dx = -filterHalfSize; dx <= filterHalfSize; dx++) {
262                     int neighborRow = row + dy;
263                     int neighborCol = col + dx;
264
265                     double pixelVal = (double)pSrc[neighborRow * numCols + neighborCol];
266                     double filterVal = filter[dy + filterHalfSize][dx + filterHalfSize];
267
268                     summa += pixelVal * filterVal;
269                 }
270             }
271
272             summa *= factor;
273             if (summa < 0) summa = 0;
274             if (summa > 255) summa = 255;
275
276             pDst[row * numCols + col] = (tByte)(summa + 0.5);
277         }
278     }
279 }
```

הפונקציה DoFiltrationByConvolution5 מבצעת קונבולוציה של תמונה בגווי אפור עם מסנן בגודל 5X5, ומאחסנת את התוצאה בתמונה חדשה לאחר הגבלת הערכים לטווח התקני [0, 255]. הפונקציה סורקת את כל הפיקסלים (למעט השוליים), מחשבת לכל אחד מהם סכום משוקלל של ערכי הפיקסלים השכנים בהתאם למסנן הנתון, מכפילה בפקטור נורמליזציה (factor), ומוודאת שהתוצאה נשארת בטווח הערכים החוקיים. היא עושה זאת באמצעות גישה ישירה לזיכרון (באמצעות מצביעים) לשם יעילות חישובית. הפונקציה שימושית במיוחד לסינון תמונות למטרות כמו חידוד, החלקה או הדגשת פרטים, בהתאם לסוג המסנן שנבחר.

Code of "CreateGaussianFilter" function

```
281
282 void CreateGaussianFilter(c2dFloatGrayImage& image, tFloat sigmaRadius)
283 {
284     tFloat x, y, r, z;
285     double summa = 0;
286     int numRows = image.NumberOfRows();
287     int numColumns = image.NumberOfColumns();
288
289     // Compute Gaussian values
290     for (int row = 0; row < numRows; row++)
291     {
292         for (int column = 0; column < numColumns; column++)
293         {
294             y = row - numRows / 2;
295             x = column - numColumns / 2;
296             r = sqrt(x * x + y * y);
297             z = exp(-(r * r) / (2 * sigmaRadius * sigmaRadius));
298             summa += z;
299             image.SetPixelValue(row, column, z);
300         }
301     }
302
303     // Normalize filter so sum = 1
304     for (int row = 0; row < numRows; row++)
305     {
306         for (int column = 0; column < numColumns; column++)
307         {
308             tFloat val = image.GetPixelValue(row, column);
309             image.SetPixelValue(row, column, val / summa);
310         }
311     }
312 }
313
```

הפונקציה CreateGaussianFilter יוצרת פילטר גאوسي דו-ממדי (2D) בגודל זהה לתמונה הנתונה (image), על בסיס פרמטר sigmaRadius שמכתיב את רמת הטשטוש.

עבור כל פיקסל, הפונקציה מחשבת את מרחקו מהמרכז, מחשבת ערך לפי פונקציית גאוס, ושומרת את הערכים בתמונה. לאחר מכן, היא מנרמלת את כל הפילטר כך שסכום כל הערכים יהיה 1, דבר הכרחי לשמירה על עוצמת התמונה לאחר קונבולוציה.

התוצאה היא פילטר רדיאלי סימטרי המתאים לעבודה בתחום התדר או לשימוש בקונבולוציה ישירה למטרת טשטוש רך.

Code of “CreateInverseGaussianFilter” function

```
315
316 void CreateInverseGaussianFilter(c2dFloatGrayImage& image, tFloat sigmaRadius)
317 {
318     const tFloat epsilon = 0.1; // Minimum allowed value to avoid division by near-zero (prevent noise amplification)
319
320     tFloat x, y, r, g, z;
321     int numRows = image.NumberOfRows(); // Image height
322     int numColumns = image.NumberOfColumns(); // Image width
323
324     // Loop over all pixels in the image
325     for (int row = 0; row < numRows; row++)
326     {
327         for (int column = 0; column < numColumns; column++)
328         {
329             // Compute (x, y) distance from the center of the image
330             y = row - numRows / 2;
331             x = column - numColumns / 2;
332
333             // Compute radial distance from center
334             r = sqrt(x * x + y * y);
335
336             // Compute Gaussian function at distance r
337             g = exp(-(r * r) / (2 * sigmaRadius * sigmaRadius));
338
339             // Clip the Gaussian to prevent division by very small values
340             if (g < epsilon)
341                 g = epsilon;
342
343             // Compute inverse of Gaussian value
344             z = 1.0 / g;
345
346             // Store the inverse filter value at current pixel
347             image.SetPixelValue(row, column, z);
348         }
349     }
350 }
```

הפונקציה CreateInverseGaussianFilter יוצרת פילטר הפוך לגאوسي ($1/\text{Gaussian}$) עבור תמונה בגודל מסוים, כאשר הפילטר מיועד לשימוש בתחום התדרים לצורך שחזור חדות של תמונה מטושטשת. הפונקציה מחשבת את ערך הגאוסיאן בכל נקודה על בסיס המרחק מהמרכז, ואז לוקחת את ההופכי שלו ($1/g$). כדי למנוע הגברה חזקה של רעש בתדרים גבוהים (שם הערכים הגאוסיים קטנים מאוד), היא מבצעת חיתוך (clipping) כך שהערך המינימלי יהיה $\epsilon = 0.1$. הפילטר שנוצר מדגיש תדרים גבוהים ובכך מסייע לשחזור מידע שאבד בטשטוש גאוזי.

Code of “ApplyFFT” function

```
354
355 // Applies the Fast Fourier Transform (FFT) to the image with a Gaussian filter in the frequency domain.
356 void ApplyFFT(c2dFloatGrayImage& imageRe, bool reverseFilter, int radius)
357 {
358     std::string filename;
359     // Load the input image
360     c2dFloatGrayImage imageIm(imageRe.NumberOfRows(), imageRe.NumberOfColumns());
361     // Create a Gaussian filter image with the same dimensions as the input image
362     c2dFloatGrayImage gaussFilter(imageRe.NumberOfRows(), imageRe.NumberOfColumns());
363     CreateGaussianFilter(gaussFilter, radius);
364     if (reverseFilter)
365     {
366         CreateInverseGaussianFilter(gaussFilter, radius);
367         filename = "gaussFilter" + std::to_string(radius) + "Reversed" + ".bmp";
368     }
369     else
370         filename = "gaussFilter" + std::to_string(radius) + ".bmp";
371     gaussFilter.SaveAsOptimalGrayBmpFile(&filename[0]);
372     // Shift both real and imaginary part images
373     ShiftHalfSize(imageRe);
374     ShiftHalfSize(imageIm);
375     // Perform Forward FFT on the images
376     DoFFT(imageRe, imageIm, FORWARD_FFT, NORMALIZE_BY_SQRT);
377     // Filter the image in the frequency domain
378     DoFiltrationInFD(imageRe, imageIm, gaussFilter);
379     // Perform Inverse FFT on the filtered images
380     DoFFT(imageRe, imageIm, REVERSE_FFT, NORMALIZE_BY_SQRT);
381     // Shift both real and imaginary part images
382     ShiftHalfSize(imageRe);
383     ShiftHalfSize(imageIm);
384 }
```

הפונקציה ApplyFFT מממשת סינון תמונה במרחב התדר באמצעות FFT. היא מבצעת המרה של התמונה לתדרים, כופלת אותה בפילטר גאوسي (לטשטוש) או בפילטר הפוך (לשחזור), ואז מחזירה את התמונה למרחב המקורי באמצעות FFT הפוך. התהליך כולל שלבים של הזזה, המרה, סינון והמרה חזרה, ומאפשר עיבוד יעיל של תמונות לצורך טשטוש או חידוד מבוסס תדר.

Code of “main” function – Task 2

```
387
388  int main()
389  {
390      //-----part 2-----//
391      c2dByteGrayImage sourceImage("Tim1.bmp");
392
393      sourceImage.Init("Tim1.bmp");
394      //Variation 1: s = 0.8 (Sharper)
395      double Filter1[] = { 0.00044, 0.02191, 0.22831, 0.49868, 0.22831, 0.02191, 0.00044 };
396      DoGaussianFiltration(sourceImage, Filter1);
397      sourceImage.SaveToGrayBmpFile("lpf1Blur.bmp");
398      sourceImage.Clear();
399
400      sourceImage.Init("Tim1.bmp");
401      //Variation 2: s = 1.0
402      double Filter2[] = { 0.00443, 0.05401, 0.24204, 0.39905, 0.24204, 0.05401, 0.00443 };
403      DoGaussianFiltration(sourceImage, Filter2);
404      sourceImage.SaveToGrayBmpFile("lpf2Blur.bmp");
405      sourceImage.Clear();
406
407      sourceImage.Init("Tim1.bmp");
408      //Variation 3: s = 1.2 (Smoother)
409      double Filter3[] = { 0.01465, 0.08312, 0.23556, 0.33335, 0.23556, 0.08312, 0.01465 };
410      DoGaussianFiltration(sourceImage, Filter3);
411      sourceImage.SaveToGrayBmpFile("lpf3Blur.bmp");
412      sourceImage.Clear();
413
414      sourceImage.Init("Tim1.bmp");
415      //Variation 4: s = 1.5 (Wider Smoothing)
416      double Filter4[] = { 0.03663, 0.11128, 0.21675, 0.27068, 0.21675, 0.11128, 0.03663 };
417      DoGaussianFiltration(sourceImage, Filter4);
418      sourceImage.SaveToGrayBmpFile("lpf4Blur.bmp");
419      sourceImage.Clear();
420
421      sourceImage.Init("Tim1.bmp");
422      //Variation 5: s = 3.5
423      double Filter5[] = { 0.11535, 0.14147, 0.15990, 0.16656, 0.15990, 0.14147, 0.11535 };
424      DoGaussianFiltration(sourceImage, Filter5);
425      sourceImage.SaveToGrayBmpFile("Tim1LPFc.bmp");
426      cout << "Tim1LPFc.bmp file was created.\n" << endl;
427      sourceImage.Clear();
428  }
```


Code of “main” function – Task 3

```
430 //-----part 3-----//
431 c2dByteGrayImage srcAfterFilter("Tim1LPFc.bmp");
432 c2dByteGrayImage dest(srcAfterFilter.NumberOfRows(), srcAfterFilter.NumberOfColumns());
433
434
435 // Define High Pass Filter kernels
436 double HPF1[3][3] = { {-2, -2, -2}, {-2, 17, -2}, {-2, -2, -2} };
437 DoFiltrationByConvolution(srcAfterFilter, dest, HPF1, 1);
438 dest.SaveToGrayBmpFile("HPF1.bmp");
439 dest.Clear();
440
441 // Define High Pass Filter kernels
442 DoFiltrationByConvolutionMINMAX("HPF1_1_MINMAX", srcAfterFilter, dest, HPF1, 1, 50);
443 DoFiltrationByConvolutionMINMAX("HPF1_2_MINMAX", srcAfterFilter, dest, HPF1, 1, 95);
444 DoFiltrationByConvolutionMINMAX("HPF1_3_MINMAX", srcAfterFilter, dest, HPF1, 1, 101);
445 DoFiltrationByConvolutionMINMAX("HPF1_4_MINMAX", srcAfterFilter, dest, HPF1, 1, 108);
446 dest.Clear();
447
448
449 double HPF2[3][3] = { {-1,-1,-1}, {-1,9,-1}, {-1,-1,-1} };
450 // Define normalization factor for HPF2
451 double HTP2factor = 1.0 / 1.0;
452 DoFiltrationByConvolution(srcAfterFilter, dest, HPF2, HTP2factor);
453 dest.SaveToGrayBmpFile("HPF2.bmp");
454 dest.Clear();
455
456
457 DoFiltrationByConvolutionMINMAX("HPF2_1_MINMAX", srcAfterFilter, dest, HPF2, HTP2factor, 50);
458 DoFiltrationByConvolutionMINMAX("HPF2_2_MINMAX", srcAfterFilter, dest, HPF2, HTP2factor, 98);
459 DoFiltrationByConvolutionMINMAX("HPF2_3_MINMAX", srcAfterFilter, dest, HPF2, HTP2factor, 115);
460 DoFiltrationByConvolutionMINMAX("HFEF1cTim1LPFc", srcAfterFilter, dest, HPF2, HTP2factor, 106);
461 cout << "HFEF1cTim1LPFc.bmp file was created.\n" << endl;
462 dest.Clear();
463
464
465 double HPF3[3][3] = { {-4,-8,-4}, {-8,64,-8}, {-4,-8,-4} };
466 // Define normalization factor for HPF3
467 double HTP3factor = 1.0 / 16.0;
468 DoFiltrationByConvolution(srcAfterFilter, dest, HPF3, HTP3factor);
469 dest.SaveToGrayBmpFile("HPF3.bmp");
470 dest.Clear();
471
472 DoFiltrationByConvolutionMINMAX("HPF3_1_MINMAX", srcAfterFilter, dest, HPF3, HTP3factor, 50);
473 DoFiltrationByConvolutionMINMAX("HPF3_2_MINMAX", srcAfterFilter, dest, HPF3, HTP3factor, 95);
474 DoFiltrationByConvolutionMINMAX("HPF3_3_MINMAX", srcAfterFilter, dest, HPF3, HTP3factor, 120);
475 DoFiltrationByConvolutionMINMAX("HPF3_4_MINMAX", srcAfterFilter, dest, HPF3, HTP3factor, 135);
476 dest.Clear();
477
```

```
479
480 double HPF4[3][3] = { {0,-1,0}, {-1,5,-1}, {0,-1,0} };
481 // Define normalization factor for HPF4
482 double HTP4factor = 1.0 / 1.0;
483 DoFiltrationByConvolution(srcAfterFilter, dest, HPF4, HTP4factor);
484 dest.SaveToGrayBmpFile("HPF4.bmp");
485 dest.Clear();
486
487 DoFiltrationByConvolutionMINMAX("HPF4_1_MINMAX", srcAfterFilter, dest, HPF4, HTP4factor, 88);
488 DoFiltrationByConvolutionMINMAX("HPF4_2_MINMAX", srcAfterFilter, dest, HPF4, HTP4factor, 95);
489 DoFiltrationByConvolutionMINMAX("HPF4_3_MINMAX", srcAfterFilter, dest, HPF4, HTP4factor, 108);
490 DoFiltrationByConvolutionMINMAX("HPF4_4_MINMAX", srcAfterFilter, dest, HPF4, HTP4factor, 116);
491
492
493 //5*5 filters
494 double HPF5[5][5] = { {-1,-1,-1,-1,-1}, {-1,-1,-1,-1,-1}, {-1,-1,25,-1,-1}, {-1,-1,-1,-1,-1}, {-1,-1,-1,-1,-1} };
495 DoFiltrationByConvolution5(srcAfterFilter, dest, HPF5, 1);
496 dest.SaveToGrayBmpFile("HPF5.bmp");
497 dest.Clear();
498 DoFiltrationByConvolution5MINMAX("HPF5_1_MINMAX", srcAfterFilter, dest, HPF5, 1, 80);
499 DoFiltrationByConvolution5MINMAX("HPF5_2_MINMAX", srcAfterFilter, dest, HPF5, 1, 100);
500 DoFiltrationByConvolution5MINMAX("HPF5_3_MINMAX", srcAfterFilter, dest, HPF5, 1, 120);
501 DoFiltrationByConvolution5MINMAX("HFEF2cTim1LPFc", srcAfterFilter, dest, HPF5, 1, 102);
502 cout << "HFEF2cTim1LPFc.bmp file was created.\n" << endl;
503 dest.Clear();
```

Code of “main” function – Task 4

```
503 dest.Clear();
504 //-----part 4-----//
505
506 ////////////// FFT blur //////////////
507
508 // Apply FFT and blur with different radii for the image "Tim1.bmp" and save the results
509 int radiusForTextImage = 100;
510 c2dFloatGrayImage imageRe;
511 imageRe.LoadFromBmpFile("Tim1.bmp");
512 ApplyFFT(imageRe, false, radiusForTextImage); // Apply FFT with blur
513 imageRe.SaveAsOptimalGrayBmpFile("BlurredFFT1.bmp"); // Save blurred image
514 imageRe.Clear(); // Clear the image for next use
515
516 radiusForTextImage = 80;
517 imageRe.LoadFromBmpFile("Tim1.bmp");
518 ApplyFFT(imageRe, false, radiusForTextImage); // Apply FFT with an even smaller blur radius
519 imageRe.SaveAsOptimalGrayBmpFile("BlurredFFT2.bmp"); // Save the image with blur
520 imageRe.Clear();
521
522 radiusForTextImage = 70;
523 imageRe.LoadFromBmpFile("Tim1.bmp");
524 ApplyFFT(imageRe, false, radiusForTextImage); // Apply FFT with an even smaller blur radius
525 imageRe.SaveAsOptimalGrayBmpFile("BlurredFFT3.bmp"); // Save the image with blur
526 imageRe.Clear();
527
528 radiusForTextImage = 60;
529 imageRe.LoadFromBmpFile("Tim1.bmp");
530 ApplyFFT(imageRe, false, radiusForTextImage); // Apply FFT with an even smaller blur radius
531 imageRe.SaveAsOptimalGrayBmpFile("BlurredFFT4.bmp"); // Save the image with blur
532 imageRe.Clear();
533
534 radiusForTextImage = 50;
535 imageRe.LoadFromBmpFile("Tim1.bmp");
536 ApplyFFT(imageRe, false, radiusForTextImage); // Apply FFT with an even smaller blur radius
537 imageRe.SaveAsOptimalGrayBmpFile("BlurredFFT5.bmp"); // Save the image with blur
538 imageRe.Clear();
539
540 radiusForTextImage = 40;
541 imageRe.LoadFromBmpFile("Tim1.bmp");
542 ApplyFFT(imageRe, false, radiusForTextImage); // Apply FFT with a smaller blur radius
543 imageRe.SaveAsOptimalGrayBmpFile("BlurredFFT6.bmp"); // Save filtered image best result
544 imageRe.Clear();
545
546 radiusForTextImage = 32;
547 imageRe.LoadFromBmpFile("Tim1.bmp");
548 ApplyFFT(imageRe, false, radiusForTextImage); // Apply FFT with a smaller blur radius
549 imageRe.SaveAsOptimalGrayBmpFile("BlurredFFT7.bmp"); // Save filtered image best result
550 imageRe.SaveAsOptimalGrayBmpFile("Tim1LPfft.bmp"); // Save filtered image best result
551 cout << "Tim1LPfft.bmp file was created.\n" << endl;
552 imageRe.Clear();
553
```



```
555 radiusForTextImage = 30;
556 imageRe.LoadFromBmpFile("Tim1.bmp");
557 ApplyFFT(imageRe, false, radiusForTextImage); // Apply FFT with a smaller blur radius
558 imageRe.SaveAsOptimalGrayBmpFile("BlurredFFT8.bmp"); // Save filtered image best result
559 imageRe.Clear();
560
561
562
563 radiusForTextImage = 20;
564 imageRe.LoadFromBmpFile("Tim1.bmp");
565 ApplyFFT(imageRe, false, radiusForTextImage); // Apply FFT with a smaller blur radius
566 imageRe.SaveAsOptimalGrayBmpFile("BlurredFFT9.bmp"); // Save filtered image best result
567 imageRe.Clear();
568
```


Code of “main” function – Task 5

```
569 //-----part 5-----//
570
571 ////////////// FFT unblur(restore) //////////////
572
573 // Apply FFT to reverse the blur effect on the "Tim1LPFfft.bmp" image with varying radii
574 radiusForTextImage = 20;
575 imageRe.LoadFromBmpFile("Tim1LPFfft.bmp");
576 ApplyFFT(imageRe, true, radiusForTextImage); // Apply FFT to unblur the image
577 imageRe.SaveAsOptimalGrayBmpFile("unBlur1.bmp"); // Save unblurred image
578
579 radiusForTextImage = 25;
580 imageRe.LoadFromBmpFile("Tim1LPFfft.bmp");
581 ApplyFFT(imageRe, true, radiusForTextImage); // Unblur with a slightly higher radius
582 imageRe.SaveAsOptimalGrayBmpFile("unBlur2.bmp"); // Save unblurred image
583 imageRe.Clear();
584
585 radiusForTextImage = 30;
586 imageRe.LoadFromBmpFile("Tim1LPFfft.bmp");
587 ApplyFFT(imageRe, true, radiusForTextImage); // Apply unblur with a higher radius
588 imageRe.SaveAsOptimalGrayBmpFile("unBlur3.bmp"); // Save result
589 imageRe.Clear();
590
591 radiusForTextImage = 35;
592 imageRe.LoadFromBmpFile("Tim1LPFfft.bmp");
593 ApplyFFT(imageRe, true, radiusForTextImage); // Continue unblurring with larger radius
594 imageRe.SaveAsOptimalGrayBmpFile("unBlur4.bmp"); // Save result
595 imageRe.Clear();
596
597 radiusForTextImage = 40;
598 imageRe.LoadFromBmpFile("Tim1LPFfft.bmp");
599 ApplyFFT(imageRe, true, radiusForTextImage); // Increase unblur radius
600 imageRe.SaveAsOptimalGrayBmpFile("unBlur5.bmp"); // Save unblurred image
601 c2dByteGrayImage Tim1RGfft(imageRe.NumberOfRows(), imageRe.NumberOfColumns());
602 minmax(200, "Tim1RGfft", imageRe, Tim1RGfft); // Process the image further
603 cout << "Tim1RGfft.bmp file was created.\n" << endl;
604 imageRe.Clear();
605
606 radiusForTextImage = 42;
607 imageRe.LoadFromBmpFile("Tim1LPFfft.bmp");
608 ApplyFFT(imageRe, true, radiusForTextImage); // Apply unblur with a larger radius
609 imageRe.SaveAsOptimalGrayBmpFile("unBlur6.bmp"); // Save the image
610 imageRe.Clear();
611
612 radiusForTextImage = 43;
613 imageRe.LoadFromBmpFile("Tim1LPFfft.bmp");
614 ApplyFFT(imageRe, true, radiusForTextImage); // Apply unblur with even more radius
615 imageRe.SaveAsOptimalGrayBmpFile("unBlur7.bmp"); // Save unblurred image
616 imageRe.Clear();
617
618
```



```
619 radiusForTextImage = 48;
620 imageRe.LoadFromBmpFile("Tim1LPFfft.bmp");
621 ApplyFFT(imageRe, true, radiusForTextImage); // Increase radius for unblur
622 imageRe.SaveAsOptimalGrayBmpFile("unBlur8.bmp"); // Save result
623 imageRe.Clear();
624
625 radiusForTextImage = 50;
626 imageRe.LoadFromBmpFile("Tim1LPFfft.bmp");
627 ApplyFFT(imageRe, true, radiusForTextImage); // Apply high radius for unblur
628 imageRe.SaveAsOptimalGrayBmpFile("unBlur9.bmp"); // Save result
629 imageRe.Clear();
630
631 radiusForTextImage = 60;
632 imageRe.LoadFromBmpFile("Tim1LPFfft.bmp");
633 ApplyFFT(imageRe, true, radiusForTextImage); // Unblur with an even larger radius
634 imageRe.SaveAsOptimalGrayBmpFile("unBlur10.bmp"); // Save image after unblur
635 imageRe.Clear();
636
637 radiusForTextImage = 70;
638 imageRe.LoadFromBmpFile("Tim1LPFfft.bmp");
639 ApplyFFT(imageRe, true, radiusForTextImage); // Apply extreme unblur
640 imageRe.SaveAsOptimalGrayBmpFile("unBlur11.bmp"); // Save the final unblurred image
641 imageRe.Clear();
642
```

Code of “main” function – Task 7 back to task 4

```
643 //-----part 7 back to task 4-----//
644
645 // Assignment 13 (TV Test Image)////
646
647 // Apply FFT and blur with varying radii for the "Tim2.bmp" image and save results
648
649 imageRe.LoadFromBmpFile("Tim2.bmp");
650 radiusForTextImage = 100;
651 ApplyFFT(imageRe, false, radiusForTextImage); // Apply blur with small radius
652 imageRe.SaveAsOptimalGrayBmpFile("blurFFTtv1.bmp");
653 imageRe.Clear();
654
655 imageRe.LoadFromBmpFile("Tim2.bmp");
656 radiusForTextImage = 80;
657 ApplyFFT(imageRe, false, radiusForTextImage); // Apply blur with moderate radius
658 imageRe.SaveAsOptimalGrayBmpFile("blurFFTtv2.bmp");
659 imageRe.Clear();
660
661 imageRe.LoadFromBmpFile("Tim2.bmp");
662 radiusForTextImage = 70;
663 ApplyFFT(imageRe, false, radiusForTextImage); // Apply blur with moderate radius
664 imageRe.SaveAsOptimalGrayBmpFile("blurFFTtv3.bmp");
665 imageRe.Clear();
666
667 imageRe.LoadFromBmpFile("Tim2.bmp");
668 radiusForTextImage = 60;
669 ApplyFFT(imageRe, false, radiusForTextImage); // Apply blur with moderate radius
670 imageRe.SaveAsOptimalGrayBmpFile("blurFFTtv4.bmp");
671 imageRe.Clear();
672
673 imageRe.LoadFromBmpFile("Tim2.bmp");
674 radiusForTextImage = 50;
675 ApplyFFT(imageRe, false, radiusForTextImage); // Apply blur with a larger radius
676 imageRe.SaveAsOptimalGrayBmpFile("blurFFTtv5.bmp");
677 imageRe.Clear();
678
679 imageRe.LoadFromBmpFile("Tim2.bmp");
680 radiusForTextImage = 40;
681 ApplyFFT(imageRe, false, radiusForTextImage); // Apply blur with another radius
682 imageRe.SaveAsOptimalGrayBmpFile("Tim2LPFfft.bmp");
683 cout << "Tim2LPFfft.bmp file was created.\n" << endl;
684 imageRe.Clear();
685
686
687
```

Code of “main” function – Task 7 back to task 5

```
687 //-----part 7 back to task 5-----//
688
689 radiusForTextImage = 20;
690 imageRe.LoadFromBmpFile("Tim2LPFFft.bmp");
691 ApplyFFT(imageRe, true, radiusForTextImage); // Reverse blur with small radius
692 imageRe.SaveAsOptimalGrayBmpFile("unblurFFTtv1.bmp");
693 imageRe.Clear();
694
695 radiusForTextImage = 25;
696 imageRe.LoadFromBmpFile("Tim2LPFFft.bmp");
697 ApplyFFT(imageRe, true, radiusForTextImage); // Reverse blur with moderate radius
698 imageRe.SaveAsOptimalGrayBmpFile("unblurFFTtv2.bmp");
699 imageRe.Clear();
700
701 radiusForTextImage = 30;
702 imageRe.LoadFromBmpFile("Tim2LPFFft.bmp");
703 ApplyFFT(imageRe, true, radiusForTextImage); // Reverse blur with moderate radius
704 imageRe.SaveAsOptimalGrayBmpFile("unblurFFTtv3.bmp");
705 imageRe.Clear();
706
707 radiusForTextImage = 35;
708 imageRe.LoadFromBmpFile("Tim2LPFFft.bmp");
709 ApplyFFT(imageRe, true, radiusForTextImage); // Reverse blur with moderate radius
710 imageRe.SaveAsOptimalGrayBmpFile("unblurFFTtv4.bmp");
711 imageRe.Clear();
712
713 radiusForTextImage = 40;
714 imageRe.LoadFromBmpFile("Tim2LPFFft.bmp");
715 ApplyFFT(imageRe, true, radiusForTextImage); // Reverse blur with larger radius
716 imageRe.SaveAsOptimalGrayBmpFile("unblurFFTtv5.bmp");
717 imageRe.SaveAsOptimalGrayBmpFile("Tim2RGfft.bmp");
718 cout << "Tim2RGfft.bmp file was created.\n" << endl;
719 imageRe.Clear();
720
721 radiusForTextImage = 42;
722 imageRe.LoadFromBmpFile("Tim2LPFFft.bmp");
723 ApplyFFT(imageRe, true, radiusForTextImage); // Reverse blur with larger radius
724 imageRe.SaveAsOptimalGrayBmpFile("unblurFFTtv6.bmp");
725 imageRe.Clear();
726
727 radiusForTextImage = 45;
728 imageRe.LoadFromBmpFile("Tim2LPFFft.bmp");
729 ApplyFFT(imageRe, true, radiusForTextImage); // Reverse blur with larger radius
730 imageRe.SaveAsOptimalGrayBmpFile("unblurFFTtv7.bmp");
731 imageRe.Clear();
732
733 radiusForTextImage = 48;
734 imageRe.LoadFromBmpFile("Tim2LPFFft.bmp");
735 ApplyFFT(imageRe, true, radiusForTextImage); // Reverse blur with a very large radius
736 imageRe.SaveAsOptimalGrayBmpFile("unblurFFTtv8.bmp");
737 imageRe.Clear();
738
```



```
741 radiusForTextImage = 50;
742 imageRe.LoadFromBmpFile("Tim2LPFFft.bmp");
743 ApplyFFT(imageRe, true, radiusForTextImage); // Reverse blur with a very large radius
744 imageRe.SaveAsOptimalGrayBmpFile("unblurFFTtv9.bmp");
745 imageRe.Clear();
746
747 radiusForTextImage = 60;
748 imageRe.LoadFromBmpFile("Tim2LPFFft.bmp");
749 ApplyFFT(imageRe, true, radiusForTextImage); // Reverse blur with a very large radius
750 imageRe.SaveAsOptimalGrayBmpFile("unblurFFTtv10.bmp");
751 imageRe.Clear();
752
753 radiusForTextImage = 70;
754 imageRe.LoadFromBmpFile("Tim2LPFFft.bmp");
755 ApplyFFT(imageRe, true, radiusForTextImage); // Reverse blur with a very large radius
756 imageRe.SaveAsOptimalGrayBmpFile("unblurFFTtv11.bmp");
757 imageRe.Clear();
758
759 // End of program
760 cout << "Press any key to exit\n" << endl;
761 return 0;
762 }
763
764
765
```

BIBLIOGRAPHY

- [1] Dr. Kosolapov, S. Lectures 08 Filtration by Convolution, 09 FFT Part 1, 10 FFT Part 2, FFT OOP, [Online], Available:
<https://sites.google.com/view/improc31651> [Accessed : July 17, 2025].
- [2] R. Fisher, S. Perkins, A. Walker and E. Wolfart. (2003). Gaussian Smoothing,[Online], Available:
<https://homepages.inf.ed.ac.uk/rbf/HIPR2/gsmooth.htm> [Accessed : July 17, 2025].
- [3] Weinhaus ,F. Digital Image Filtering,[Online], Available:
http://www.fmwconcepts.com/imagemagick/digital_image_filtering.pdf [Accessed : July 17, 2025].

WHAT DID WE LEARNED

- **The Challenges of Image Restoration**

I learned that restoring a blurred image is much more complex than simply applying the inverse of the blur. Different high-frequency enhancing convolution filters produce varying results, and fine-tuning their parameters requires multiple iterations. This taught me the importance of experimenting and analyzing the trade-offs between sharpening and noise amplification.

- **Practical Application of Fourier Transforms in Image Processing**

Implementing Gaussian filtering in the frequency domain using FFT deepened my understanding of how convolution in the spatial domain relates to multiplication in the frequency domain. I also realized that designing an effective inverse filter is not straightforward, as directly applying $1/\text{Gaussian}$ can amplify noise and artifacts.

- **Efficient Memory Management and Pointer-Based Image Processing in C++**

Accessing image pixels using pointers instead of arrays significantly improved my understanding of memory management in C++. This experience reinforced the importance of working efficiently with image data structures and choosing the appropriate data types (unsigned char, integer, or float) to balance precision and performance.